

SAPIENZA
UNIVERSITÀ DI ROMA

Big Data Computing

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente
Prof. Gabriele Tolomei

Appunti di
Alessio Cavedal
GitHub: https://github.com/AlessioCavedal

Anno Accademico 2026/2027

Indice

Il corso **ii**

I Infrastruttura per i big data **1**

1 Il paradigma scale-out e lo storage distribuito **2**
1.1 Perché scalare in orizzontale 2
1.2 Principi di progetto dei file system distribuiti 4
1.3 Google File System 7
1.4 HDFS 11
1.5 Località dei dati e replicazione 14
1.6 Gestione dei guasti 16

2 Il modello di esecuzione MapReduce **21**
2.1 L'astrazione MapReduce 21
2.2 Architettura di esecuzione 23
2.3 Shuffle, sort e reduce 26
2.4 Tolleranza ai guasti e straggler 28
2.5 Limiti di MapReduce 31

3 Apache Spark e i grafi di dataflow **32**
3.1 Oltre la pipeline a due stadi 32
3.2 Resilient Distributed Dataset 34
3.3 Trasformazioni e azioni 35
3.4 Valutazione pigra e DAG di esecuzione 37
3.5 Dipendenze strette e ampie 39
3.6 Lineage e tolleranza ai guasti 41
3.7 Limiti degli RDD 43

Il corso

Il corso è diviso in quattro parti.

1. **Infrastruttura per i big data** (*Big Data Infrastructure*): storage scale-out, tolleranza ai guasti tramite *lineage*, MapReduce, esecuzione a DAG in memoria (Spark), *streaming*.
2. **Rappresentazioni dei dati ad alta dimensionalità** (*High-Dimensional Data Representations*): geometria degli spazi ad alta dimensione, riduzione della dimensionalità (Johnson–Lindenstrauss), layout colonnari, *embedding* di testi, immagini e grafi.
3. **Problemi classici senza apprendimento** (*Classical Non-Learning Tasks*): indicizzazione su scala web, teoria del campionamento, *Approximate Query Processing*, *sketch* sub-lineari (HyperLogLog, Count-Min), *Locality-Sensitive Hashing*.
4. **Sistemi di apprendimento scale-out** (*Scale-Out Learning Systems*): ERM su larga scala, SGD parallelo e asincrono, *Parameter Server*, *Federated Learning*, ottimizzazione della banda di memoria.

Parte I

Infrastruttura per i big data

Capitolo 1

Il paradigma scale-out e lo storage distribuito

Una macchina infinitamente veloce e affidabile non esiste: il calcolo sui big data comincia dall'accettare questo fatto e dal progettare attorno a esso.

1.1 Perché scalare in orizzontale

I sistemi per i big data esistono perché i volumi di dati superano ciò che una singola macchina può memorizzare o elaborare. Qualche esempio:

- i *crawl* del web, cioè le raccolte di pagine scaricate dai motori di ricerca: centinaia di miliardi di pagine, petabyte di HTML grezzo;
- log e telemetria: dati di sensori e *clickstream* (le sequenze di clic degli utenti), che si accumulano al ritmo di terabyte al giorno;
- dati scientifici e aziendali: genomica, astronomia, log delle transazioni delle grandi imprese, fino alla scala degli exabyte.

Se un solo disco, una sola macchina e un solo punto di guasto non bastano a contenere e servire questi dati, quale alternativa strutturale esiste?

1.1.1 I limiti della singola macchina

I dati sono cresciuti molto più in fretta della capacità di qualunque macchina. La capacità dei dischi raddoppia ogni qualche anno, i dati ogni qualche mese; alcuni carichi, come gli indici del web o le reti di sensori, producono dati più in fretta di quanto un singolo disco riesca anche solo a scriverli.

Un server, per quanto potente, ha poi un tetto fisico su ogni risorsa:

- **CPU**: un numero finito di core per socket e di socket per scheda madre;
- **RAM**: un numero finito di slot DIMM e di memoria indirizzabile;
- **disco**: un numero finito di alloggiamenti per i dischi e una banda di I/O finita per controller;
- **rete**: la banda finita delle schede di rete, in entrata e in uscita.

Una macchina più grande sposta il tetto più in alto, ma non lo elimina.

1.1.2 Scalabilità verticale

Definizione (scalabilità verticale). La **scalabilità verticale** (*vertical scaling*, o *scale up*) consiste nel rendere più grande una singola macchina: più core, più RAM, dischi più veloci o più capienti sullo stesso nodo.

Il vantaggio è la semplicità: spesso il software non va toccato, perché la macchina semplicemente fa di più. Il problema è fin dove si può arrivare, e a che prezzo.

Il primo limite è il costo, che nella fascia alta cresce molto più che linearmente (Tabella 1.1): l'hardware di pregio (più socket, interconnessioni speciali, memorie specializzate) si paga carissimo, e oltre una certa soglia raddoppiare la capacità costa ben più del doppio.

Tabella 1.1. Capacità e costo relativi delle configurazioni di fascia alta.

Configurazione	Capacità relativa	Costo relativo
Server commodity	1×	1×
Server di fascia alta	4×	oltre 10×
Classe mainframe, top di gamma	16×	oltre 100×

Il secondo limite è fisico: una scheda madre ha un numero massimo di socket e di slot DIMM, e una macchina ha un unico involucro di alimentazione e raffreddamento. A un certo punto nessuna cifra compra una macchina più grande per un dato carico. La scalabilità verticale fa guadagnare tempo, ma non è una strategia di lungo periodo per dati su scala web o exabyte.

1.1.3 Scalabilità orizzontale

Definizione (scalabilità orizzontale). La **scalabilità orizzontale** (*horizontal scaling*, o *scale out*) consiste nell'aggiungere macchine invece di ingrandirle: il carico si distribuisce su molti nodi indipendenti, ciascuno di capacità modesta, e la capacità del *cluster* cresce aggiungendo nodi, non potenziandone uno.

È la strategia alla base di praticamente tutti i sistemi per i big data. L'idea chiave dietro sistemi come GFS e HDFS è costruire i cluster con macchine **commodity**, cioè economiche, standard e inaffidabili, invece che con poche macchine costose e affidabili:

- migliaia di nodi economici possono costare, nel complesso, meno di pochi nodi di pregio;
- il guasto del singolo nodo diventa un evento previsto, non un'eccezione;
- l'affidabilità diventa responsabilità del sistema, non dell'hardware.

La Tabella 1.2 mette a confronto i due approcci.

Tabella 1.2. Scalabilità verticale e orizzontale a confronto.

Aspetto	Verticale (*scale up*)	Orizzontale (*scale out*)
Approccio	una macchina più grande	più macchine
Tetto	limite fisico rigido	di fatto illimitato
Crescita del costo	superlineare nella fascia alta	circa lineare (hardware commodity)
Guasti	affidabilità dell'hardware	tolleranza ai guasti nel software
Complessità	bassa (stesso modello software)	alta (coordinamento distribuito)

1.1.4 Il guasto come evento normale

Su un cluster il guasto hardware smette di essere un caso limite. Con migliaia di dischi, statisticamente c'è sempre qualche disco che si sta rompendo; le reti si partizionano, le macchine vanno in crash, interi rack restano senza alimentazione. Un sistema progettato nell'ipotesi che i guasti siano rari, a questa scala, fallirebbe di continuo: l'infrastruttura per i big data deve trattare il guasto come un evento di routine.

Scalando in orizzontale cambiano anche le voci di costo che contano:

- **costo per byte** (storage): i dischi commodity rendono economico lo spazio grezzo, e il costo dominante diventa quello della replicazione e della durabilità;
- **costo per FLOP** (calcolo): le CPU commodity rendono economico il calcolo grezzo, e il costo dominante diventa quello del coordinamento e dello spostamento dei dati.

In entrambi i casi, a scala, il collo di bottiglia non è più il costo dell'hardware, ma la capacità del sistema di coordinare tante parti economiche.

1.2 Principi di progetto dei file system distribuiti

Definizione (file system distribuito). Un **file system distribuito** (*Distributed File System*, DFS) offre un'unica interfaccia di file system, sostenuta da uno storage sparso su molte macchine. Ai client i file appaiono come se stessero in un solo file system logico; in realtà il loro contenuto è suddiviso, replicato e distribuito su un cluster.

Il DFS nasconde questa complessità dietro le solite operazioni sui file (`open`, `read`, `write`, `close`). L'obiettivo è uno storage da petabyte che, per le applicazioni, sembri un normale file system, solo molto grande.

1.2.1 Obiettivi e ipotesi di progetto

I DFS per i big data sono ottimizzati per un carico preciso, e tre scelte ne determinano l'architettura.

Throughput prima della latenza. Il sistema massimizza il *throughput*, cioè la quantità totale di dati trasferiti al secondo in tutto il cluster, e tollera che la singola operazione impieghi più tempo che su un disco locale. L'analisi batch di petabyte ha bisogno di banda aggregata, non di tempi di risposta al millisecondo per la singola richiesta.

File enormi, letti in sequenza. I file sono di solito enormi, da gigabyte a terabyte ciascuno, e si leggono per lo più in sequenza, dall'inizio alla fine o a grandi pezzi. Le piccole letture e scritture casuali non sono il carico di riferimento.

Write-once, read-many. Un file viene scritto una volta sola, per esempio con una serie di *append* (aggiunte in coda), poi viene chiuso e letto molte volte da job di analisi diversi. La modifica sul posto del contenuto esistente è rara o vietata, così il sistema non deve supportare in modo efficiente aggiornamenti arbitrari.

1.2.2 Blocchi di dimensione fissa

> **Definizione (blocco).** Un DFS non memorizza un file come un'unica unità contigua, ma lo divide in **blocchi** (*blocks*, o *chunks*) di dimensione fissa. Ogni blocco viene memorizzato, replicato e gestito in modo indipendente, e blocchi diversi dello stesso file possono stare su macchine diverse.

È la decisione strutturale più importante nel progetto di uno storage distribuito. Fissare la dimensione dei blocchi, invece di usare frammenti di lunghezza variabile, semplifica tutto il sistema:

- i **metadati**: il sistema tiene traccia di dove stanno i blocchi, non di intervalli di byte arbitrari;
- il **bilanciamento del carico**: i blocchi sono unità di lavoro intercambiabili, facili da collocare e da spostare;
- la **replicazione**: ogni blocco si replica per conto suo, in modo uniforme.

Resta da decidere quanto farli grandi (Tabella 1.3). I DFS per i big data scelgono blocchi grandi, da decine a centinaia di megabyte: l'opposto dei file system locali tradizionali, che usano blocchi di pochi kilobyte.[1]

[1] Lo svantaggio per i file piccoli non sta tanto nello spazio su disco: sia GFS sia HDFS salvano ogni blocco come un file del file system locale, che occupa solo lo spazio effettivamente scritto. Il costo sta altrove: ogni file e ogni blocco pesa sulla memoria del nodo che gestisce i metadati, e un file piccolo sta in pochissimi blocchi, quindi le poche macchine che lo ospitano diventano punti caldi (*hot spot*) se molti client lo leggono insieme.

Tabella 1.3. Il compromesso sulla dimensione dei blocchi.

Blocchi	Vantaggi	Svantaggi
Piccoli (es. 4 KB)	granularità fine, poco spreco per i file piccoli	metadati enormi su larga scala
Grandi (es. 64–128 MB)	pochi metadati, I/O sequenziale efficiente	costosi per i file piccoli, bilanciamento del carico più grossolano

1.2.3 Metadati separati dai dati

Una decisione centrale è separare il sistema che sa «dove stanno le cose» da quello che conserva i byte veri e propri:

- il **servizio dei metadati** gestisce il **namespace**, cioè la struttura gerarchica di directory e file, e sa dove si trova ogni blocco;
- i **server dei dati** conservano il contenuto dei blocchi sui propri dischi locali.

Così ciascuno dei due sottosistemi si ottimizza per conto suo: il primo per le ricerche veloci, i secondi per il throughput di I/O grezzo. Il servizio dei metadati mantiene due corrispondenze: da file a blocchi (quali blocchi compongono ciascun file) e da blocchi a posizioni (quali server ospitano le repliche di ogni blocco). Le tiene soprattutto in memoria, perché su scala la velocità delle ricerche è critica per le prestazioni.

Molti DFS, GFS e HDFS compresi, affidano i metadati a un **unico nodo coordinatore**.

- **Vantaggi**: una vista del namespace semplice e globalmente consistente, e nessun bisogno di consenso distribuito a ogni modifica dei metadati.
- **Svantaggi**: il coordinatore è un potenziale punto di guasto singolo (*single point of failure*) e un potenziale collo di bottiglia, se le operazioni sui metadati crescono troppo.

Il coordinatore, poi, non vede passare i dati. Il client gli chiede solo dove si trova un blocco, riceve l'elenco delle posizioni e scambia i dati direttamente con i server dei dati. Le interrogazioni dei metadati, piccole e frequenti, restano così separate dai trasferimenti dei dati, grandi e pesanti in banda, e il coordinatore non diventa un collo di bottiglia.

1.2.4 Topologia di rete e replicazione

Gli storage distribuiti conoscono la topologia fisica della rete, non solo l'identità logica delle macchine. Le macchine sono raggruppate in **rack**, ciascuno con il proprio switch (*top-of-rack switch*), e la banda all'interno di un rack è di solito molto più alta di quella tra rack diversi. Le decisioni di posizionamento e di scheduling tengono conto di questa gerarchia: macchina, rack, datacenter.

Su una singola macchina, contro il guasto di un disco basta il RAID. Su un cluster non più: il RAID protegge dal guasto di un disco, non da quello di una macchina, di

un rack o della rete, e con migliaia di macchine il numero atteso di guasti in ogni istante non è zero. È il DFS stesso a dover replicare i dati su macchine diverse, e possibilmente su rack diversi. Qui la **replicazione** non è un'ottimizzazione: è il meccanismo principale di durabilità.

1.3 Google File System

Il **Google File System** (GFS) è stato progettato per i carichi interni di Google: memorizzare ed elaborare enormi quantità di pagine web, di log e di file di indice, su migliaia di macchine commodity in cui il guasto è la norma. Segue le tre ipotesi della Sezione 1.2: file enormi, letture sequenziali, scritture fatte soprattutto di append. È il diretto antenato di HDFS.

1.3.1 Architettura

Un cluster GFS ha un solo **master**, che coordina, e molti **chunkserver**, che memorizzano i dati e li servono direttamente ai client (Figura 1.1).

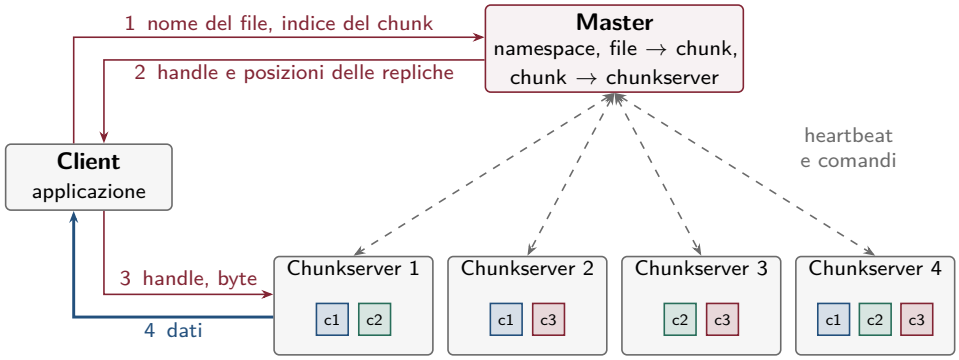


Figura 1.1. Architettura di GFS e percorso di una lettura. In rosso i messaggi di controllo, in blu il flusso dei dati, tratteggiati heartbeat e comandi. Ogni chunk (c1, c2, c3) ha tre repliche su chunkserver diversi. HDFS ha la stessa struttura, con NameNode e DataNode al posto di master e chunkserver.

Il master. È il coordinatore unico della Sezione 1.2, con i suoi vantaggi e i suoi rischi:

- mantiene il namespace del file system, cioè la struttura delle directory e la corrispondenza tra file e chunk;
- mantiene la mappa delle posizioni: quali chunkserver ospitano quali chunk;
- assegna i *lease*, che regolano le scritture;
- coordina la creazione, la replicazione e la *garbage collection* dei chunk.

Tutti questi metadati stanno in memoria, così il master risponde molto in fretta. Le posizioni dei chunk, anzi, non vengono nemmeno salvate su disco: all'avvio il master le ricostruisce interrogando i chunkserver. Il prezzo è un limite: la dimensione complessiva del namespace è vincolata dalla memoria del master.

I chunkserver. Sono i cavalli da lavoro di GFS. Ogni chunkserver salva i chunk come normali file sul proprio file system locale (Linux), serve direttamente ai client le letture e le scritture dei chunk che possiede e comunica periodicamente al master il proprio inventario.

Chunk da 64 MB. Chunk così grandi significano pochi metadati sul master, poche richieste del client al master durante una lettura lunga e trasferimenti grandi e sequenziali. Lo svantaggio, già visto, riguarda i file piccoli: un compromesso accettato, dato il carico a cui GFS è destinato.

Replicazione. Per default ogni chunk è replicato 3 volte, su chunkserver diversi e tenendo conto dei rack, così sopravvive al guasto di un singolo chunkserver o disco. Il fattore di replicazione si può configurare file per file, e i dati più critici possono avere più copie.

Heartbeat. Master e chunkserver dialogano con gli **heartbeat** (letteralmente «battiti cardiaci»): messaggi periodici con cui ogni chunkserver segnala di essere vivo e riporta quali chunk possiede. Servono a più cose:

- il master sa quali chunkserver sono attivi e che cosa contengono, senza dover salvare su disco le posizioni dei chunk;
- confrontando gli inventari, riconosce i chunk sotto-replicati, dopo un guasto, e quelli sovra-replicati, dopo un recupero;
- nella risposta all'heartbeat include i propri comandi, come «replica questo chunk altrove» o «cancella quest'altro».

1.3.2 Lettura

Una lettura procede così (i numeri sono quelli della Figura 1.1):

1. dall'offset richiesto il client calcola l'indice del chunk (l'offset diviso per 64 MB, arrotondato per difetto) e chiede al master il *chunk handle*, cioè l'identificativo del chunk, e l'elenco dei chunkserver che lo ospitano;
2. il master risponde con handle e posizioni, che il client tiene in cache per un breve periodo;
3. il client contatta direttamente la replica più vicina;
4. il chunkserver trasmette i dati richiesti direttamente al client.

Il master interviene solo nei passi 1 e 2: i dati non passano mai da lui.

1.3.3 Scrittura e lease

Quando più client modificano lo stesso chunk, tutte le repliche devono applicare le modifiche nello stesso ordine. GFS non fa decidere l'ordine al master per ogni scrittura, e non usa un protocollo di consenso: delega la decisione a una delle repliche, tramite un lease.

Definizione (lease). Un **lease** (letteralmente «locazione») è una concessione a tempo con cui il master designa, per un certo chunk, una delle repliche come **primaria**. Finché il lease è valido, la primaria decide l'ordine seriale di tutte le modifiche (*mutations*) concorrenti di quel chunk; le altre repliche, le **secondarie**, le applicano in quell'ordine.

Il lease ha una scadenza, ma può essere prolungato finché la primaria sta gestendo delle modifiche. Se la primaria diventa irraggiungibile, il master aspetta che il vecchio lease scada e solo dopo ne concede uno nuovo a un'altra replica: la vecchia primaria potrebbe essere solo lenta, non morta, e così non ci sono mai due primarie che ordinano le scritture in modo diverso.

La scrittura procede in cinque passi (Figura 1.2):

1. il client chiede al master quali chunkserver ospitano le repliche del chunk e quale di esse detiene il lease, cioè è la primaria;
2. il client invia (*push*) i dati a tutte le repliche, che li tengono in un buffer: li manda al chunkserver più vicino, che li inoltra al più vicino tra i restanti, e così via, in catena;
3. il client manda la richiesta di scrittura alla primaria;
4. la primaria assegna alla scrittura un numero d'ordine e la inoltra alle secondarie, che la applicano in quell'ordine. Se arrivano insieme le richieste di due client, prima B e poi A, la primaria dà 1 a B e 2 ad A, e ogni replica applica B e poi A, qualunque sia l'ordine in cui le erano arrivati i dati;
5. le secondarie confermano alla primaria, e la primaria risponde al client quando tutte le repliche hanno confermato.

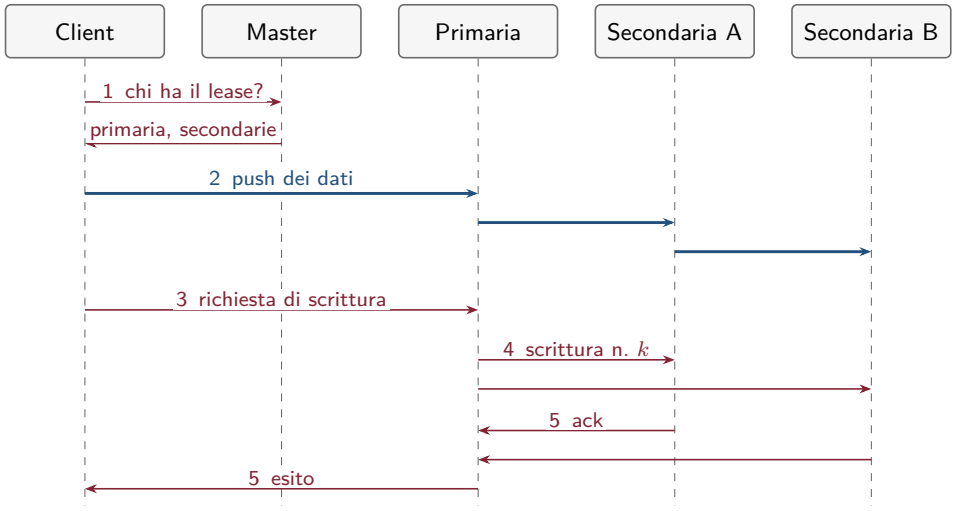


Figura 1.2. Scrittura in GFS. Il flusso dei dati (in blu) è separato dal flusso di controllo (in rosso): i dati arrivano prima, in catena, a tutte le repliche, poi la primaria decide in che ordine applicarli.

Il flusso dei dati (il push) e il flusso di controllo (l'ordine delle scritture) sono separati di proposito, per le prestazioni: il trasferimento pesante non aspetta le decisioni sull'ordine, e l'ordine si decide con messaggi piccoli.

Consistenza. Poiché tutte le repliche applicano le modifiche nell'ordine della primaria, le repliche di un chunk restano identiche: la regione del file è **consistent** (coerente), cioè tutti i client vedono gli stessi dati, da qualunque replica leggano. Non è detto però che sia **defined** (definita), cioè che contenga per intero ciò che una scrittura ha scritto. Una scrittura a cavallo di due chunk viene divisa in due operazioni, una per chunk, e ciascuna è ordinata dalla primaria del suo chunk. Se A e B scrivono insieme la stessa regione, la primaria del primo chunk può applicare A e poi B, quella del secondo B e poi A: alla fine il primo chunk contiene la parte di B, il secondo quella di A. Le repliche sono identiche, ma la regione mescola metà di B e metà di A. Con un solo scrittore questo non succede, e la regione è anche defined. GFS accetta questa garanzia più debole in cambio delle prestazioni.

1.3.4 Cancellazione e garbage collection

GFS non libera subito lo spazio di un file cancellato. Il file viene rinominato con un nome nascosto e conservato per qualche giorno; lo spazio dei suoi chunk viene recuperato più tardi, in modo pigro (*lazy*), da un processo di **garbage collection** che lavora durante le normali comunicazioni tra master e chunkserver. La cancellazione diventa più semplice e robusta: una cancellazione accidentale si può annullare entro il periodo di conservazione, e non serve un'operazione sincrona su tutto il cluster.

1.3.5 Tolleranza ai guasti del master

Il master tiene i metadati in memoria, ma deve sopravvivere ai crash:

- ogni modifica del namespace viene aggiunta in coda a un **operation log** (registro delle operazioni) persistente, replicato su più macchine;
- periodicamente il master scrive un **checkpoint**, un'istantanea compatta del proprio stato, per non dover rieseguire da capo, al riavvio, un log sempre più lungo;
- al riavvio carica l'ultimo checkpoint e riesegue le voci del log successive.

GFS usa anche degli **shadow master**: repliche in sola lettura dello stato del master, che applicano continuamente lo stesso operation log. Possono rispondere alle interrogazioni di sola lettura sui metadati, alleggerendo il master, ma se il master cade non ne prendono automaticamente il posto: la sostituzione richiede una procedura separata di recupero o di promozione.

1.3.6 Limiti

Il master unico, per quanto semplice, ha limiti noti:

- **scalabilità dei metadati**: l'intero namespace deve stare nella memoria di una macchina;

- **collo di bottiglia**: un ritmo molto alto di operazioni sui metadati, per esempio con moltissimi file piccoli, può sovraccaricare il master;
- **failover non istantaneo**: anche con checkpoint e log, riprendersi da un crash del master richiede tempo.

Sono proprio questi limiti a motivare le evoluzioni successive, come la Federation di HDFS.

1.4 HDFS

L'**Hadoop Distributed File System** (HDFS) è lo storage open source ispirato direttamente a GFS, nato nel progetto Apache Hadoop per l'elaborazione batch su larga scala, all'inizio proprio per MapReduce. Ne adotta l'architettura: un nodo che gestisce i metadati, molti nodi che memorizzano i dati, blocchi grandi, replicazione. Cambia soprattutto la terminologia.

1.4.1 NameNode e DataNode

Il **NameNode** fa ciò che fa il master di GFS: tiene in memoria il namespace, con i permessi, e le corrispondenze tra file, blocchi e DataNode, e coordina replicazione e ribilanciamento. I **DataNode** sono i chunkserver di HDFS: salvano i blocchi come file del proprio file system locale e servono direttamente ai client letture e scritture. I blocchi sono di 128 MB per default e si possono configurare, di solito fino a 256 MB, contro i 64 MB fissi di GFS.

Heartbeat e block report. In GFS l'heartbeat portava anche l'inventario dei chunk; HDFS separa i due compiti in due messaggi periodici:

- gli **heartbeat**, frequenti (ogni pochi secondi), sono un semplice segnale di vita: «sono attivo»;
- i **block report**, meno frequenti, contengono l'elenco completo dei blocchi presenti sul DataNode.

Insieme danno al NameNode una vista accurata, quasi in tempo reale, della distribuzione dei blocchi e della salute dei nodi.

1.4.2 FsImage, EditLog e Secondary NameNode

Il NameNode rende persistenti i metadati con due strutture, che corrispondono al checkpoint e all'operation log di GFS:

- la **FsImage**, un'istantanea completa del namespace in un certo momento;
- l'**EditLog**, un log in sola aggiunta (*append-only*) di tutte le modifiche del namespace (creazioni, cancellazioni, rinomine...) successive all'ultima FsImage.

Al riavvio funzionano come in GFS: il NameNode carica l'ultima FsImage e riesegue le voci dell'EditLog successive; poco dopo prende una nuova FsImage, così al riavvio seguente il log da rieseguire è corto.

Il Secondary NameNode non è un NameNode di riserva. Il nome inganna: il **Secondary NameNode** non è una replica pronta a subentrare (*hot standby*), e non risponde nemmeno alle letture sui metadati, come fanno gli shadow master di GFS. Il suo compito è fondere periodicamente FsImage ed EditLog in un nuovo checkpoint e rimandarlo al NameNode, togliendo questo lavoro, potenzialmente costoso, dal percorso critico del NameNode. Per avere un vero failover serve la configurazione ad alta affidabilità.

1.4.3 Alta affidabilità e Federation

Per eliminare il punto di guasto singolo, le installazioni di produzione usano coppie di NameNode in **High Availability** (HA):

- un NameNode *Active* gestisce tutte le richieste dei client;
- uno o più NameNode *Standby* restano sincronizzati, pronti a subentrare;
- un edit log condiviso e consistente, spesso affidato a un servizio di logging separato basato sul consenso, tiene aggiornati gli Standby;
- un meccanismo di failover automatico rileva il guasto dell'Active e promuove uno Standby.

Contro il collo di bottiglia dei metadati, lo stesso limite visto per GFS, HDFS offre la **Federation** (federazione): più NameNode indipendenti, ciascuno responsabile di una porzione distinta del namespace. I DataNode si registrano presso tutti i NameNode della federazione e ospitano blocchi per ciascuno di essi. Ogni NameNode ha ancora il proprio limite di memoria, ma la capacità complessiva del namespace cresce con il numero di NameNode.

1.4.4 Lettura e scrittura

La lettura segue lo schema di GFS; in più il NameNode restituisce i DataNode di ogni blocco ordinati per vicinanza al client.

La scrittura invece usa una **pipeline** di repliche (Figura 1.3):

1. il client chiede al NameNode di creare un nuovo file; il NameNode controlla i permessi e alloca il primo blocco;
2. il NameNode restituisce l'elenco dei DataNode che formano la pipeline di replicazione del blocco;
3. il client invia il flusso dei dati al primo DataNode della pipeline;
4. ogni DataNode inoltra i dati al successivo, in sequenza;
5. le conferme (*ack*) risalgono la pipeline quando tutti i DataNode hanno reso persistenti i dati.

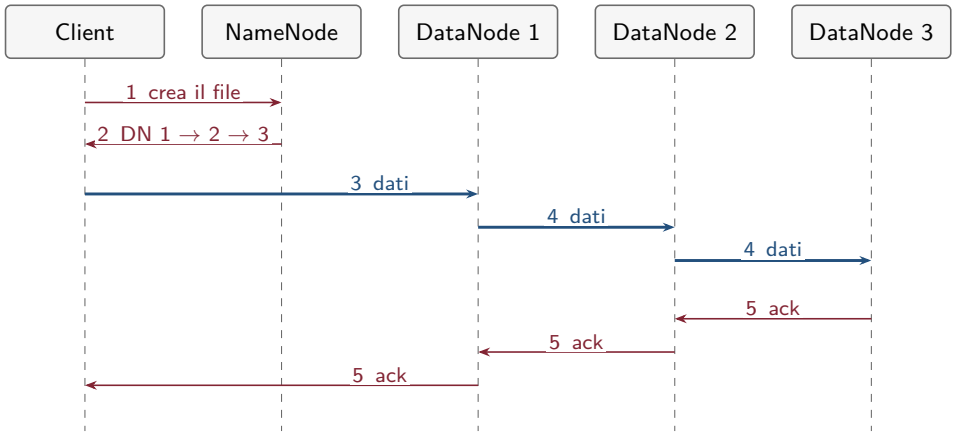


Figura 1.3. Scrittura in HDFS: i dati attraversano una sola volta la catena di DataNode, e le conferme la risalgono all'indietro.

Il percorso dei dati è lo stesso di GFS, una catena di repliche; la differenza sta nel controllo. In GFS più client possono scrivere nello stesso chunk, e serve una primaria con il lease che decida l'ordine, separata dal flusso dei dati. In HDFS un file ha un solo scrittore alla volta e si può solo estendere in coda, secondo l'ipotesi write-once, read-many (Sezione 1.2): l'ordine è quello in cui i dati attraversano la pipeline. La Tabella 1.4 raccoglie le corrispondenze tra i due sistemi.

Tabella 1.4. GFS e HDFS: terminologia e differenze principali.

Concetto	GFS	HDFS
Nodo dei metadati	master	NameNode
Nodo dei dati	chunkserver	DataNode
Unità di memorizzazione	chunk da 64 MB, fissi	blocco da 128 MB per default, configurabile
Persistenza dei metadati	checkpoint + operation log	FsImage + EditLog
Nodo di supporto	shadow master: risponde alle letture sui metadati	Secondary NameNode: fonde i checkpoint, non risponde ai client
Scrittura	più scrittori per chunk, ordine deciso dalla primaria con il lease	un solo scrittore per file, pipeline di DataNode
Alta affidabilità	shadow master, senza failover automatico	NameNode Standby con failover automatico; Federation

1.5 Località dei dati e replicazione

1.5.1 Spostare il calcolo, non i dati

Definizione (località dei dati). Il principio di **località dei dati** (*data locality*) dice che, su scala big data, portare il calcolo dove i dati già risiedono costa molto meno che portare i dati dove si fa il calcolo. Lo scheduler esegue quindi ogni elaborazione sulla macchina che ha già i dati che le servono, o su una vicina.

Il motivo è che la banda di rete tra macchine è di ordini di grandezza inferiore a quella dei dischi e della memoria locali (Tabella 1.5), e spostare in rete terabyte di dati per ogni job sarebbe proibitivamente lento e costoso. Lo strato di storage rende possibile il principio: GFS e HDFS sanno dove si trova fisicamente ogni blocco, lo scheduler può eseguire il calcolo su quella macchina, e in rete viaggiano solo i risultati, molto più piccoli dei dati grezzi.

Tabella 1.5. Bande indicative in un cluster tipico.

Risorsa	Banda indicativa
Lettura da disco locale	da centinaia di MB/s a qualche GB/s
Rete nello stesso rack	circa 1–10 Gb/s, condivisi da tutto il rack
Rete tra rack	meno, condivisa sulla dorsale (*backbone*) del datacenter

Nel leggere la tabella conviene notare due cose: la banda di rete è in gigabit al secondo (10 Gb/s sono circa 1,25 GB/s), e soprattutto è condivisa da tutte le macchine del rack, mentre ogni macchina ha i propri dischi. L'I/O locale è veloce e «gratuito»; l'I/O di rete, specie tra rack, è la risorsa scarsa che la località dei dati serve a risparmiare.

I blocchi si prestano a questo schema. Ogni blocco si elabora in modo in gran parte indipendente dagli altri, quindi molti blocchi si elaborano in parallelo, ciascuno sulla macchina che lo ospita; e la replicazione rende lo storage affidabile, così lo strato di calcolo può darlo per scontato. È la struttura che sfrutterà MapReduce: un task di map per blocco, eseguito vicino ai dati del blocco.

1.5.2 Posizionamento delle repliche

Dove collocare le repliche di un blocco non si decide a caso: bisogna bilanciare due obiettivi in conflitto.

- **Durabilità**: le repliche vanno sparse su domini di guasto diversi, cioè su macchine diverse e possibilmente su rack diversi.
- **Efficienza**: sparpagliarle troppo aumenta il traffico tra rack durante le scritture.

Il compromesso comune è il posizionamento **rack-aware**: alcune repliche nello stesso rack, e almeno una in un rack diverso. Con fattore 3, due repliche condividono un rack, dove la replicazione è veloce, e la terza sta in un altro, così il blocco sopravvive

anche alla perdita di un intero rack, per esempio per un guasto dell'alimentazione o dello switch che il rack condivide.

La politica di default di HDFS, con fattore 3, è la seguente (Figura 1.4):

1. la prima replica va sul DataNode su cui scrive il client, o su un nodo vicino;
2. la seconda va su un nodo di un rack diverso da quello della prima;
3. la terza va su un altro nodo dello stesso rack della seconda.

La durabilità tra rack è garantita, perché la prima replica sta in un rack e le altre due nell'altro; il traffico tra rack in scrittura resta limitato, perché le repliche 2 e 3 condividono un solo passaggio da un rack all'altro.

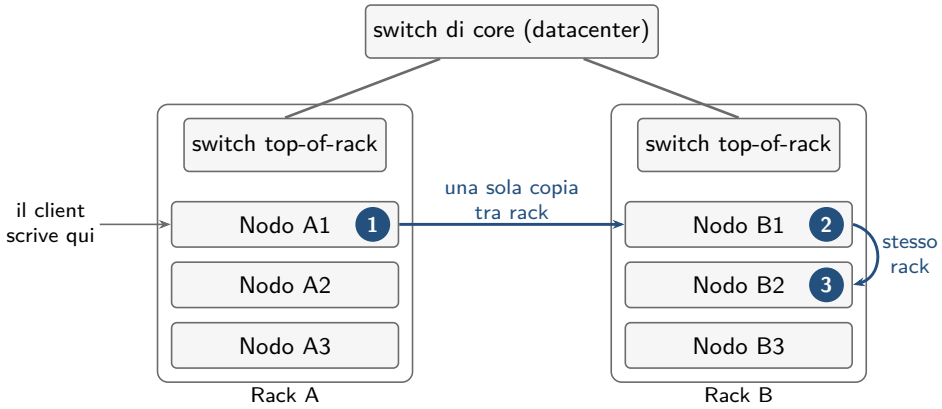


Figura 1.4. Posizionamento di default delle repliche in HDFS, con fattore 3. Una sola copia attraversa la rete tra i rack; se si guasta un rack intero, nell'altro resta almeno una replica.

HDFS però non deduce da solo la topologia della rete: è l'amministratore a doverla configurare, con uno script o una tabella che associa ogni DataNode al proprio rack. Senza questa configurazione HDFS tratta tutti i nodi come se fossero in un unico rack, e perde i benefici del posizionamento tra rack.

1.5.3 Fattore di replicazione

Definizione (fattore di replicazione). Il **fattore di replicazione** r è il numero di copie mantenute per ogni blocco.

Aumentarlo migliora la durabilità, ma costa in spazio e in scritture: ogni scrittura deve raggiungere tutte le repliche prima di essere confermata, come si è visto per GFS e per HDFS, quindi più repliche significano più traffico di rete e più tempo prima che una scrittura sia considerata durevole. Fattori più alti del default si usano in modo selettivo, per dati particolarmente critici o letti molto spesso.

Esempio (probabilità di perdere un chunk). Supponiamo che ogni nodo si guasti con probabilità annua $p = 0{,}02$, indipendentemente dagli altri. Un chunk replicato su r nodi va perso solo se si guastano tutte le sue repliche, nella stessa

breve finestra di tempo. Per eventi indipendenti le probabilità si moltiplicano: con $r = 2$ la perdita ha probabilità $0{,}02 \times 0{,}02 = 0{,}0004$, e in generale

$$P(\text{perdita}) \approx p^r.$$

Ogni replica in più moltiplica la probabilità per p, cioè la divide per $1/p = 50$, quasi due ordini di grandezza (Tabella 1.6).[2] Il calcolo però vale solo se i guasti sono indipendenti: è proprio l'indipendenza che il posizionamento rack-aware cerca di garantire.

Tabella 1.6. Fattore di replicazione: spazio, guasti tollerati e probabilità di perdita, con $p = 0{,}02$.

Fattore r	Spazio occupato	Guasti tollerati	$P(\text{perdita}) \approx p^r$
1 (nessuna replica)	1×	nessuno	0,02
2	2×	1	0,0004
3 (default)	3×	2 simultanei	0,000008

1.5.4 Domini di guasto e bilanciamento

La replicazione deve tenere conto dei guasti correlati, che colpiscono insieme ambiti via via più ampi, i **domini di guasto** (*failure domains*):

- il **disco**: si perdono le copie che stavano su quel disco;
- il **nodo**: si perdono tutti i chunk della macchina;
- il **rack**: si perdono tutti i nodi che condividono lo switch o l'alimentazione;
- il **datacenter**: si perde un'intera struttura, per un blackout, un guasto di rete o un disastro.

Distribuire le repliche su questi domini protegge da guasti di scala via via più grande.

Oltre a replicare, il sistema cerca di bilanciare l'occupazione dello spazio, per non sovraccaricare alcuni nodi mentre altri restano semivuoti: sia il posizionamento dei nuovi chunk sia la ri-replicazione tengono conto dell'utilizzo attuale dei dischi. Un cluster ben bilanciato evita i punti caldi, che diventerebbero colli di bottiglia o punti di guasto precoce.

1.6 Gestione dei guasti

La qualità di uno storage distribuito si misura soprattutto da come degrada e si riprende, non da quanto raramente si guasta.

[2] Il modello è volutamente grossolano: p è una probabilità annua, mentre la perdita richiede che le repliche si guastino nella stessa breve finestra di tempo. I numeri mostrano l'effetto dell'esponente r, non stimano il rischio reale.

1.6.1 Quanti guasti aspettarsi

I guasti dei dischi si misurano con il **tasso annuo di guasto** (*Annualized Failure Rate*, AFR), cioè la probabilità che un disco si guasti nel corso di un anno. Per i dischi commodity sta di solito tra l'1% e il 5%, a seconda di età, modello e condizioni d'uso. Per un disco solo è poco; per un cluster no.

Consideriamo un cluster con

$$N = 10\,000 \text{ dischi}, \qquad \text{AFR} = 2\% = 0{,}02,$$

e sia p la probabilità che un certo disco si guasti durante una settimana: qui p è settimanale, non annua come nell'esempio sulla replicazione. Se i guasti sono distribuiti in modo uniforme sulle 52 settimane dell'anno,

$$p \approx \frac{0{,}02}{52} \approx 0{,}0003846.$$

Il numero di guasti è una variabile aleatoria. Per il disco i si definisce la variabile indicatrice

$$X_i = \begin{cases} 1 & \text{se il disco } i \text{ si guasta durante la settimana,} \\ 0 & \text{altrimenti.} \end{cases}$$

Vale 1 con probabilità p: si scrive $X_i \sim \text{Bernoulli}(p)$, cioè «$X_i$ ha distribuzione di Bernoulli di parametro p». Il numero totale di guasti nella settimana è la somma $X = \sum_{i=1}^{N} X_i$, che conta gli 1. Se i guasti dei dischi sono indipendenti e hanno tutti la stessa probabilità p (variabili i.i.d., indipendenti e identicamente distribuite), X ha distribuzione **binomiale**, quella del numero di successi in N prove indipendenti:

$$X \sim \text{Binomial}(N, p), \qquad N = 10\,000, \quad p \approx 0{,}0003846.$$

Probabilità di k guasti. Perché si guastino esattamente k dischi, k dischi devono guastarsi, con probabilità p^k, e gli altri $N - k$ no, con probabilità $(1-p)^{N-k}$; i k dischi si possono scegliere in $\binom{N}{k}$ modi (il coefficiente binomiale, «N su k»):

$$P(X = k) = \binom{N}{k} p^k (1-p)^{N-k}.$$

Per esempio, con $k = 4$ i modi di scegliere i 4 dischi sono circa $4{,}16 \cdot 10^{14}$, la probabilità che si guastino proprio quei 4 è $p^4 \approx 2{,}19 \cdot 10^{-14}$ e quella che gli altri 9 996 reggano è $(1-p)^{9996} \approx 0{,}0214$:

$$P(X = 4) \approx 4{,}16 \cdot 10^{14} \times 2{,}19 \cdot 10^{-14} \times 0{,}0214 \approx 0{,}195,$$

quasi il 20%.

Valore atteso e varianza. Il **valore atteso** $\mathbb{E}[X]$ (la E viene da *expected value*) è la media di X sul lungo periodo. Ogni disco contribuisce in media p guasti a settimana, quindi N dischi ne contribuiscono Np:

$$\mathbb{E}[X] = Np = 10\,000 \times \frac{0{,}02}{52} \approx 3{,}85.$$

In media il cluster perde quasi 4 dischi a settimana. Il valore 3,85 non è uno che X possa assumere, perché X è un intero: è la media sul lungo periodo. Quanto X oscilla attorno alla media lo dice la **varianza** $\mathrm{Var}(X)$, la media dei quadrati degli scarti dalla media. Per una binomiale vale $Np(1-p)$, perché ogni disco ha varianza $p(1-p)$ e per variabili indipendenti le varianze si sommano:

$$\mathrm{Var}(X) = 10\,000 \cdot 0{,}0003846 \cdot (1 - 0{,}0003846) \approx 3{,}84.$$

La sua radice, la **deviazione standard** $\sigma_X = \sqrt{\mathrm{Var}(X)} \approx 1{,}96$, riporta lo scarto alla stessa unità di X: in una settimana tipica il cluster perde $3{,}85 \pm 1{,}96$ dischi, cioè di solito tra 2 e 6.

Almeno un guasto. Qual è la probabilità che in una settimana si guasti almeno un disco? Invece di sommare le probabilità di $X = 1, 2, \ldots$, conviene passare al complemento: nessun disco si guasta se ognuno degli N regge, con probabilità $1 - p$ ciascuno, quindi $P(X = 0) = (1-p)^N \approx 0{,}021$ e

$$P(X \geq 1) = 1 - (1-p)^N = 1 - (1 - 0{,}0003846)^{10\,000} \approx 0{,}979.$$

Con queste ipotesi semplificate, un cluster di 10 000 dischi ha almeno un guasto nel 98% circa delle settimane. Un tasso annuo del 2% sembra piccolo per un disco, ma su 10 000 dischi diventa un flusso regolare di guasti, circa 4 a settimana: è l'effetto di scala che giustifica la filosofia del «guasto normale».

1.6.2 Tipi di guasto

Non tutti i guasti sono uguali, e il sistema deve gestirne di diversi tipi:

- **transitori**: brevi interruzioni di rete o pause momentanee di un processo; il nodo sta bene, ma per un po' non risponde;
- **permanenti**: un disco che muore, un guasto hardware, un nodo dismesso; il nodo o i dati sono persi per sempre;
- **correlati**: quelli che colpiscono un intero dominio di guasto, come un rack.

1.6.3 Rilevamento: heartbeat e timeout

GFS e HDFS rilevano i guasti con lo stesso meccanismo: ogni nodo dei dati manda heartbeat periodici al nodo dei metadati e, se non arriva nessun heartbeat entro un certo *timeout*, il nodo viene considerato morto. È intrinsecamente un compromesso, e ricorre spesso nei sistemi distribuiti (Tabella 1.7): un timeout troppo breve rischia falsi positivi, cioè nodi sani ma lenti dati per morti, mentre uno troppo lungo

ritarda la scoperta dei guasti veri. Non esiste un valore giusto in assoluto: si regola sull'affidabilità della rete e sul carico del cluster specifico.

Tabella 1.7. Il compromesso sul timeout degli heartbeat.

Timeout	Rilevamento	Rischio
Breve	rapido	falsi positivi su nodi lenti ma vivi; ri-replicazioni inutili
Lungo	meno falsi positivi	risposta lenta ai guasti veri; finestra più lunga con ridondanza ridotta

1.6.4 Recupero

Guasto di un nodo dei dati. Quando un chunkserver o un DataNode viene dato per morto, il recupero segue sempre lo stesso schema:

1. il nodo dei metadati rileva il guasto dagli heartbeat mancati;
2. individua tutti i chunk o blocchi che stavano su quel nodo e li segna come sotto-replicati;
3. pianifica la ri-replicazione di ciascuno, copiandolo da una replica superstite su un nodo sano; la copia avviene in background, con la priorità di ripristinare la ridondanza il più in fretta che la rete e i dischi permettono;
4. alla fine il sistema torna al fattore di replicazione pieno, in automatico, senza intervento manuale.

Guasto del nodo dei metadati. Recuperare il master o il NameNode è più delicato, perché contiene l'unica vista autorevole del namespace. Senza alta affidabilità il cluster resta fermo finché il nodo non viene riavviato e ricarica checkpoint e log (Sezione 1.3.5); con l'alta affidabilità subentra uno Standby (Sezione 1.4), con un'interruzione minima. Per questo le installazioni di produzione la usano quasi sempre.

Corruzione silenziosa e checksum. Non tutti i guasti sono crash evidenti: un disco può restituire bit leggermente corrotti senza segnalare alcun errore. GFS e HDFS salvano un **checksum**, un codice breve calcolato dal contenuto, insieme a ogni chunk o blocco, e lo verificano a ogni lettura: se non corrisponde, i dati sono corrotti. Una corruzione scoperta fa scattare lo stesso meccanismo di ri-replicazione: si legge da un'altra replica, presumibilmente integra, e si ricrea quella corrotta. Così il sistema si ripara da solo: per i guasti di routine non serve alcun intervento umano.

Esempio (un chunkserver muore durante una lettura). Un caso concreto, dall'inizio alla fine:

1. un client sta leggendo un chunk dal chunkserver A, che va in crash a metà trasferimento;

2. la richiesta del client va in timeout o restituisce un errore;
3. il client chiede di nuovo al master le posizioni delle altre repliche dello stesso chunk;
4. il client ripete la lettura da un altro chunkserver, B o C: per l'applicazione la lettura riesce, in modo trasparente per l'utente finale;
5. separatamente, il master si accorge degli heartbeat mancati di A e pianifica la ri-replicazione di tutti i suoi chunk altrove.

I passi dal 2 al 4 riguardano il client; il passo 5 avviene del tutto in background.

Capitolo 2

Il modello di esecuzione MapReduce

Lo storage del capitolo precedente non calcola nulla: i dati stanno in blocchi replicati su migliaia di macchine, e qualcuno deve elaborarli. Prima di MapReduce ogni job su larga scala doveva risolvere da capo gli stessi problemi difficili: come dividere il lavoro tra le macchine, come gestire una macchina che va in crash a metà job, come raccogliere e combinare i risultati parziali. **MapReduce** li risolve una volta per tutte in un framework riusabile: il programmatore scrive codice dall'aspetto sequenziale, e il sistema si occupa di parallelismo, scheduling e guasti. Non inventa algoritmi nuovi, ma un modo di eseguire due operazioni semplicissime, map e reduce, in modo sicuro su migliaia di macchine inaffidabili.

2.1 L'astrazione MapReduce

2.1.1 Radici funzionali

MapReduce prende due primitive dalla programmazione funzionale:

- `map(f, list)` applica la funzione `f` a ogni elemento di una lista, in modo indipendente, e produce una nuova lista: se `f` eleva al quadrato, su `[1,2,3]` dà `[1,4,9]`;
- `reduce(g, list)` combina gli elementi di una lista a due a due con la funzione `g`, e produce un unico risultato aggregato: se `g` è la somma, su `[1,2,3,4]` calcola $((1 + 2) + 3) + 4 = 10$.

Il punto cruciale è che map lavora su elementi indipendenti, senza alcun ordine o dipendenza tra loro: è esattamente questo che la rende parallelizzabile su più macchine.

2.1.2 Il modello di programmazione

Definizione (job MapReduce). Dal punto di vista del programmatore, scrivere un **job MapReduce** significa implementare soltanto due funzioni:

```
map(key, value)
    → list(intermediate_key, intermediate_value)
reduce(intermediate_key, list(intermediate_value))
    → list(output_value)
```

Il programmatore non scrive mai codice di scheduling, di rete o di gestione dei guasti: è il framework, non l'utente, a distribuire le chiamate di map e di reduce sul cluster.

La funzione **map** elabora un record di input alla volta, in modo indipendente:

- riceve una singola coppia (chiave, valore) dell'input;
- emette zero, una o molte coppie (chiave, valore) **intermedie**;
- non vede gli altri record dell'input, e non può dipendere dallo stato di altre chiamate di map.

Questa assenza di stato permette al framework di eseguire milioni di chiamate di map in parallelo, in qualunque ordine e su qualunque macchina.

La funzione **reduce** elabora tutti i valori associati a una chiave intermedia, dopo che sono stati raggruppati:

- riceve una chiave intermedia e la lista completa dei valori emessi per quella chiave da tutte le chiamate di map;
- emette il valore o i valori finali per quella chiave.

Chiavi diverse si elaborano in modo indipendente, quindi anche le chiamate di reduce possono girare in parallelo.

2.1.3 Due esempi

Esempio (word count). L'esempio canonico: contare quante volte compare ogni parola in un'enorme collezione di documenti, per esempio salvati come blocchi HDFS. L'output desiderato è, per ogni parola distinta, il numero totale di occorrenze in tutti i documenti.

```
map(document_id, document_text):
    for each word w in document_text:
        emit(w, 1)

reduce(word, list_of_counts):
    total = sum(list_of_counts)
    emit(word, total)
```

Ogni chiamata di map elabora un documento e, per ogni parola che incontra, emette la coppia `(w,1)`, come a dire «ho visto questa parola una volta»: map non conta ancora nulla. Ogni chiamata di reduce riceve una parola con la lista

di tutti gli 1 emessi per lei in tutti i documenti, e li somma. È il framework a garantire che tutti gli 1 di una parola arrivino alla stessa chiamata di reduce.

Il problema è banale, ma mette in gioco tutti i meccanismi di MapReduce, e nient'altro. L'astrazione però non serve solo a contare.

Esempio (indice invertito). L'**indice invertito** di un motore di ricerca associa a ogni parola la lista dei documenti che la contengono.

```
map(document_id, document_text):
    for each word w in document_text:
        emit(w, document_id)

reduce(word, list_of_document_ids):
    emit(word, unique_sorted(list_of_document_ids))
```

La struttura è la stessa del word count: cambiano solo i valori emessi da map e la logica di aggregazione di reduce. È qui la vera forza dell'astrazione.

2.1.4 Determinismo

La tolleranza ai guasti di MapReduce poggia su un'ipotesi critica.

Definizione (funzione pura). Le funzioni map e reduce devono essere **pure**: a parità di input producono sempre lo stesso output, e non hanno **effetti collaterali** (*side effects*), cioè non scrivono su stati esterni e non dipendono da stati esterni modificabili.

Se un task fallisce e viene semplicemente rieseguito altrove, il determinismo garantisce che la nuova esecuzione produca risultati identici: è questo che rende «basta riprovare» una strategia valida di tolleranza ai guasti. Se map e reduce non fossero deterministiche, una riesecuzione potrebbe produrre in silenzio risultati diversi da quelli dell'esecuzione originale, ormai persa. Per questo, in MapReduce, il requisito di determinismo non è negoziabile.

2.2 Architettura di esecuzione

Trasformare le funzioni map e reduce dell'utente in un job che gira su un cluster richiede alcuni passi coordinati (Figura 2.1):

1. dividere i dati di input in pezzi indipendenti (*input split*);
2. eseguire un map task per ogni pezzo, distribuendoli sulle macchine disponibili;
3. partizionare l'output intermedio e smistarlo verso i reduce task giusti: è lo *shuffle*;
4. eseguire i reduce task, ciascuno responsabile di un sottoinsieme delle chiavi intermedie;

5. raccogliere l'output finale.

Un **map task** elabora un pezzo di input ed emette coppie intermedie; un **reduce task** elabora tutti i valori di una partizione delle chiavi intermedie.

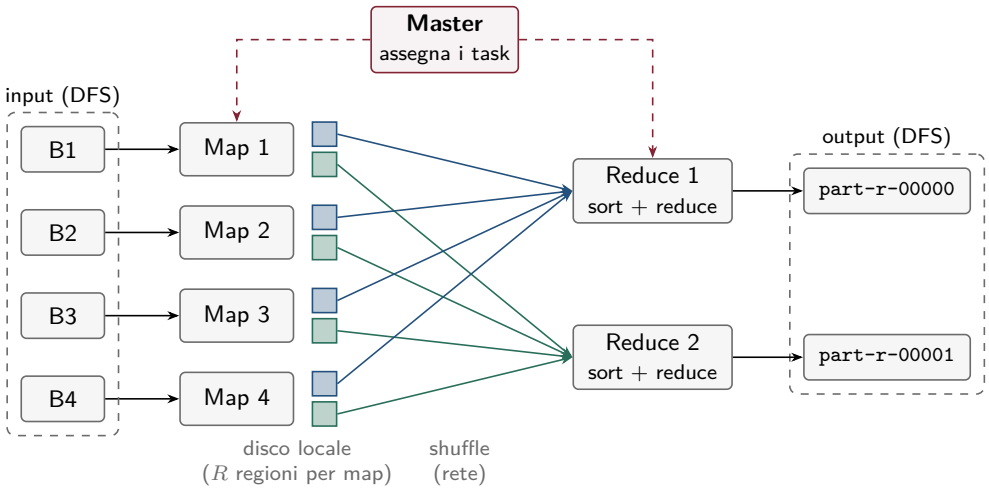


Figura 2.1. Esecuzione di un job con $M = 4$ map task e $R = 2$ reduce task. Ogni map task legge un blocco e scrive il proprio output sul disco locale, già diviso in R regioni (in blu quella per il reduce 1, in verde quella per il reduce 2). Nello shuffle ogni reduce task preleva la propria regione da tutti i map task; poi ordina, applica reduce e scrive un file di output nel DFS.

2.2.1 Il master

Un coordinatore centrale, chiamato **master** o *JobTracker*, gestisce l'esecuzione del job:

- divide l'input in pezzi adatti ai map task;
- assegna map task e reduce task alle macchine libere, i **worker**;
- segue l'avanzamento dei task e li riassegna in caso di guasto.

Concettualmente ha un ruolo analogo al master di GFS o al NameNode di HDFS, ma per il calcolo invece che per lo storage.

2.2.2 Map task e località dei dati

L'input di MapReduce arriva di solito direttamente dal file system distribuito del Capitolo 1, dove è già diviso in blocchi (HDFS) o in chunk (GFS). Per default si crea un map task per ogni blocco di input: l'unità naturale di lavoro parallelo è ereditata dallo strato di storage.

Lo scheduling applica il principio di località dei dati (Sezione 1.5): il master esegue ogni map task sulla macchina che ospita già il blocco corrispondente, o vicino a essa. Così si riduce al minimo il traffico di rete della fase map, che legge di gran lunga il volume più grande dell'intero job. Se nessuna macchina perfettamente locale è disponibile, il master ripiega su una macchina dello stesso rack, e solo in ultima istanza su una macchina qualsiasi.

Il numero di map task è di solito molto più grande del numero di macchine: un task per blocco, e i job grandi hanno migliaia di blocchi. Un cluster di, diciamo, 100 macchine esegue queste migliaia di task in molte **ondate** (*waves*) successive. Questa sovra-decomposizione (*over-decomposition*), con tanti task piccoli per macchina, aiuta il bilanciamento del carico: una macchina lenta o guasta trattiene solo qualche task piccolo, che si rilancia in fretta altrove.

Output intermedio. L'output di un map task non va direttamente a destinazione: viene scritto sul **disco locale** della macchina che ha eseguito il task, non nel file system distribuito. Questi dati intermedi servono solo temporaneamente, alla fase reduce, e scriverli in locale evita un inutile costo di replicazione.

2.2.3 Partizionamento

Prima che i reduce task possano partire, le coppie intermedie vanno raggruppate per chiave e instradate verso il reduce task giusto. Se ne occupa una **funzione di partizionamento**, che associa ogni chiave intermedia a uno degli R reduce task del job: tutte le coppie con la stessa chiave devono arrivare allo stesso reduce task, qualunque sia il map task che le ha prodotte.

La strategia più comune usa una funzione hash:

```
partition(key) = hash(key) mod R
```

`mod` è il resto della divisione: con $R = 2$ vale 0 o 1, e sceglie uno dei due reduce task. Se l'hash di «the» vale 17, ogni map task calcola lo stesso resto, 1, e tutte le coppie di «the» finiscono nello stesso reduce task. Con una funzione hash ragionevolmente uniforme le chiavi si distribuiscono in modo più o meno equo; per esigenze particolari di bilanciamento si possono fornire funzioni di partizionamento personalizzate.

2.2.4 Numero di reduce task

A differenza del numero di map task, che dipende dal numero di blocchi di input, il numero R di reduce task lo sceglie di solito l'utente, cercando un compromesso (Tabella 2.1). Una regola pratica comune è sceglierlo dell'ordine del numero di worker disponibili, a volte un suo piccolo multiplo.

Tabella 2.1. Il compromesso sul numero di reduce task.

Reduce task	Conseguenza
Troppo pochi	ognuno gestisce una quota enorme delle chiavi: poco parallelismo, e il rischio di un singolo reducer sovraccarico («caldo»)
Troppi	più file di output da gestire, più costo di coordinamento nello shuffle, benefici decrescenti oltre il numero di macchine disponibili

2.2.5 Combiner

Definizione (combiner). Un **combiner** è un'ottimizzazione facoltativa: un piccolo reduce eseguito in locale sull'output di ciascun map task, prima dello shuffle. Applica la stessa logica di aggregazione della funzione reduce, spesso proprio la stessa funzione, quando questa è associativa e commutativa.

Il combiner riduce il volume dei dati intermedi che devono attraversare la rete durante lo shuffle. Nel word count, per esempio, somma localmente le coppie `(w,1)` prodotte dallo stesso map task: se una parola compare mille volte nel blocco, in rete viaggia una sola coppia `(w,1000)` invece di mille coppie `(w,1)`. Serve che l'aggregazione sia associativa e commutativa perché il risultato non deve cambiare sommando prima i parziali di ogni map task, in qualunque ordine: $(1+1+1)+(1+1)$ fa 5 come $1+1+1+1+1$.

2.2.6 Output in shard

Come l'input arriva in tanti blocchi, anche l'output viene scritto in tanti file, detti **shard** (frammenti), e non in un unico file monolitico: ogni reduce task scrive il proprio, con nomi convenzionali come `part-r-00000`, `part-r-00001` e così via. Un grande dataset si rappresenta naturalmente come un insieme di file, e i job successivi possono leggere direttamente questi shard come proprio input distribuito.

2.3 Shuffle, sort e reduce

2.3.1 Lo shuffle

Definizione (shuffle). Lo **shuffle** (letteralmente «rimescolamento») è il trasferimento in rete dell'output intermedio dei map task verso i reduce task giusti. Ogni reduce task deve ricevere tutte le coppie intermedie delle chiavi che gli spettano, qualunque sia la macchina che le ha prodotte.

In generale i dati possono dover viaggiare da ogni map task a ogni reduce task: è uno schema di comunicazione **all-to-all** (da tutti a tutti). La preparazione però comincia prima di qualunque trasferimento. Mentre emette le coppie intermedie, ogni map task applica subito la funzione di partizionamento a ciascuna chiave, e divide il proprio output sul disco locale in R **regioni**, una per reduce task, già ordinate o raggruppate per destinazione. In seguito ogni reduce task preleva (*pull*) da ciascun map task solo la regione che gli spetta (Figura 2.1).

2.3.2 Perché lo shuffle costa

Lo shuffle è considerato la parte più costosa di un job MapReduce, per più ragioni:

- con M map task e R reduce task, può richiedere fino a $M \times R$ trasferimenti distinti: nella Figura 2.1, $4 \times 2 = 8$;

- a differenza della fase map, non può sfruttare la località: in generale i dati devono attraversare la rete, perché non c'è modo di rendere ogni reduce task «locale» a tutti i map task contemporaneamente;
- se D è la dimensione complessiva dei dati intermedi, il traffico totale in rete è dell'ordine di D: il parallelismo non lo riduce, lo distribuisce soltanto su tanti trasferimenti più piccoli;
- anche il disco pesa: i dati intermedi vengono scritti sul disco locale dai map task, riletti per essere trasferiti, e spesso riscritti e ordinati dal lato dei reduce.

In molti job reali è lo shuffle, non il calcolo di map o di reduce, a dominare il tempo totale di esecuzione.

Due tecniche pratiche ne riducono l'impatto:

- i **combiner**, che riducono il volume dei dati intermedi prima ancora che debbano essere trasferiti;
- la **compressione** dei dati intermedi prima del trasferimento, che scambia tempo di CPU con meno I/O di rete e di disco: su un cluster lo scambio di solito conviene.

2.3.3 Ordinamento e fase reduce

Dopo aver ricevuto le proprie partizioni da tutti i map task, ogni reduce task ordina i dati per chiave intermedia, così tutti i valori della stessa chiave finiscono uno accanto all'altro. Di solito il framework fa questo ordinamento mentre fonde i dati arrivati dai diversi map task, con un procedimento simile al merge sort.

L'ordinamento è una scelta deliberata per semplificare il lavoro del programmatore: la funzione reduce può contare sul fatto che tutti i valori di una chiave arrivano insieme, in un unico gruppo contiguo, e non deve implementare da sé alcuna logica di raggruppamento.

Con i dati ordinati e raggruppati per chiave, il reduce task elabora un gruppo alla volta:

```
for each (key, sorted_list_of_values) in sorted_input:
    call reduce(key, sorted_list_of_values)
    emit result to output file
```

Ogni chiamata della funzione reduce dell'utente gestisce esattamente una chiave, con la lista completa dei suoi valori. Se una chiave ha un numero enorme di valori, cioè se c'è **sbilanciamento dei dati** (*data skew*), l'uso di memoria può diventare un problema.

Finito il lavoro, il reduce task scrive il proprio output nel file system distribuito, a differenza dell'output dei map task, che restava sul disco locale. Ogni reduce task produce il proprio shard, che viene replicato come qualunque altro file del DFS.

Esempio (word count dall'inizio alla fine). Si segue il word count su due documenti minuscoli, Doc A «the cat sat» e Doc B «the dog ran», con 2 reduce task e partizionamento hash (Figura 2.2).

- **Map**. Il map task di Doc A emette `(the,1)`, `(cat,1)`, `(sat,1)`; quello di Doc B emette `(the,1)`, `(dog,1)`, `(ran,1)`.
- **Partizionamento**. Supponiamo che l'hash mandi «the» e «cat» al reduce 1, e «sat», «dog» e «ran» al reduce 2: sul disco locale di ogni map task le coppie finiscono nella regione del reduce corrispondente.
- **Shuffle e sort**. Il reduce 1 riceve, già ordinate, `(cat,[1])` e `(the,[1,1])`; il reduce 2 riceve `(dog,[1])`, `(ran,[1])` e `(sat,[1])`.
- **Reduce**. Il reduce 1 emette `(cat,1)` e `(the,2)`; il reduce 2 emette `(dog,1)`, `(ran,1)` e `(sat,1)`.

«The» compare in entrambi i documenti, ma le due copie arrivano allo stesso reduce task: è esattamente ciò che la funzione di partizionamento garantisce.

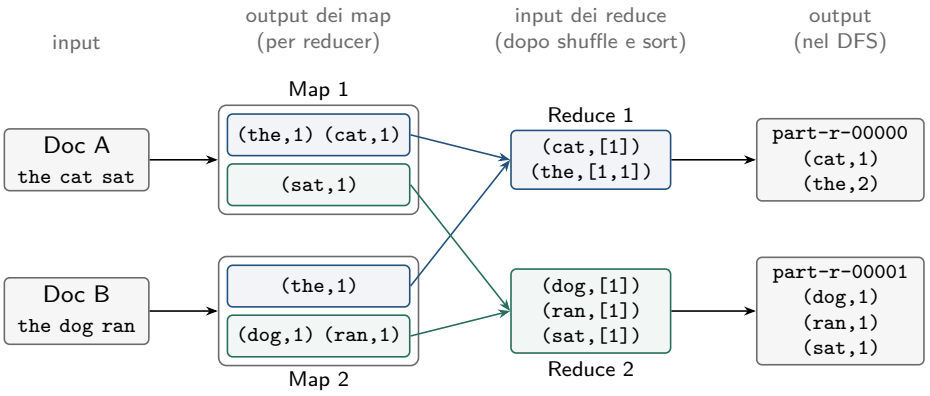


Figura 2.2. Word count su due documenti con 2 reduce task: in blu le chiavi del reduce 1, in verde quelle del reduce 2. Le due coppie `(the,1)`, prodotte da map task diversi, arrivano allo stesso reduce task.

2.4 Tolleranza ai guasti e straggler

Il Capitolo 1 trattava i guasti dello storage: dischi, chunkserver, master. MapReduce aggiunge una seconda dimensione, i guasti durante il calcolo:

- un worker può andare in crash mentre esegue un map task o un reduce task;
- un task può bloccarsi o procedere in modo anomalo, lentissimo, senza fallire davvero;
- il master che coordina il job può a sua volta guastarsi.

2.4.1 Guasto di un worker

Il master rileva i guasti dei worker con lo stesso meccanismo dello strato di storage: i worker gli mandano heartbeat periodici con l'avanzamento dei propri task, e se un worker manca gli heartbeat oltre un certo timeout il master lo considera guasto. A

quel punto vanno gestiti i task che il worker stava eseguendo, e in certi casi anche quelli che aveva già completato (Tabella 2.2).

Map task. Il master riassegna semplicemente il map task a un altro worker sano. Lo fa anche se il task originale era già terminato, in parte o del tutto: il suo output stava sul disco locale della macchina guasta, che ora è irraggiungibile. Poiché la funzione map è deterministica (Sezione 2.1), la nuova esecuzione produce un output identico.

Reduce task. Il recupero è simile, ma basta ripercorrere lo shuffle, non il calcolo dei map: il master riassegna il reduce task a un altro worker, che preleva di nuovo i dati intermedi che gli spettano dagli output dei map task, ancora disponibili. Gli output dei map non vanno ricalcolati, solo riletti. Un reduce task già completato, invece, non va rieseguito: il suo output sta già nel file system distribuito.

Tabella 2.2. Che cosa si riesegue quando un worker si guasta.

Task	In corso	Completato
Map	rieseguito su un altro worker	rieseguito: l'output stava sul disco locale del worker guasto
Reduce	rieseguito, rileggendo gli output dei map	non rieseguito: l'output è già nel DFS

2.4.2 Guasto del master

Anche il master che coordina il job può guastarsi. Nel progetto originale di MapReduce il guasto del master è raro, perché si tratta di un unico processo che segue il job, e lo si gestisce riavviando l'intero job da capo. È un compromesso deliberato a favore della semplicità: il nodo dei metadati di GFS o HDFS deve sopravvivere per tutta la vita del cluster, mentre il master di MapReduce deve sopravvivere solo per la durata di un job.

2.4.3 Straggler ed esecuzione speculativa

> **Definizione (straggler).** Uno **straggler** (letteralmente «ritardatario») è un task ancora in esecuzione, non andato in crash, ma molto più lento di altri task equivalenti.

Uno straggler non fa scattare il rilevamento dei guasti basato sugli heartbeat, perché il worker è vivo e risponde. Eppure, senza contromisure, il tempo di completamento dell'intero job è vincolato dal task più lento, anche se 999 task su 1000 hanno finito in fretta. Le cause di un task sano ma lento sono diverse:

- **degrado dell'hardware**: un disco o una CPU che rende meno di quanto dovrebbe, senza guastarsi del tutto;

- **contesa delle risorse**: altri job o processi che competono per CPU, disco o rete sulla stessa macchina;
- **sbilanciamento dei dati**: un reduce task a cui tocca un numero insolitamente grande di valori per una chiave, e che impiega quindi un tempo sproporzionato.

La soluzione di MapReduce è l'**esecuzione speculativa** (*speculative execution*):

1. verso la fine del job, il master individua i task molto più lenti della media dei task simili;
2. per ciascuno lancia una **copia di backup** su un altro worker, libero;
3. la prima copia che termina, originale o backup, fornisce il risultato; l'altra viene scartata.

Esempio (l'effetto su un job). Un job ha 1000 map task che impiegano circa 10 secondi ciascuno, ma uno di essi, per un hardware degradato, ne impiega 200 (Figura 2.3). Senza speculazione il job finisce dopo circa 200 secondi, vincolato dallo straggler. Con la speculazione, quando gli altri task hanno finito il master lancia una copia di backup, che impiega anch'essa circa 10 secondi: il job finisce in circa 20 secondi, 10 per gli altri task più 10 per la copia. Un solo straggler può allungare il job di un ordine di grandezza, ed è proprio questo il caso che l'esecuzione speculativa colpisce.

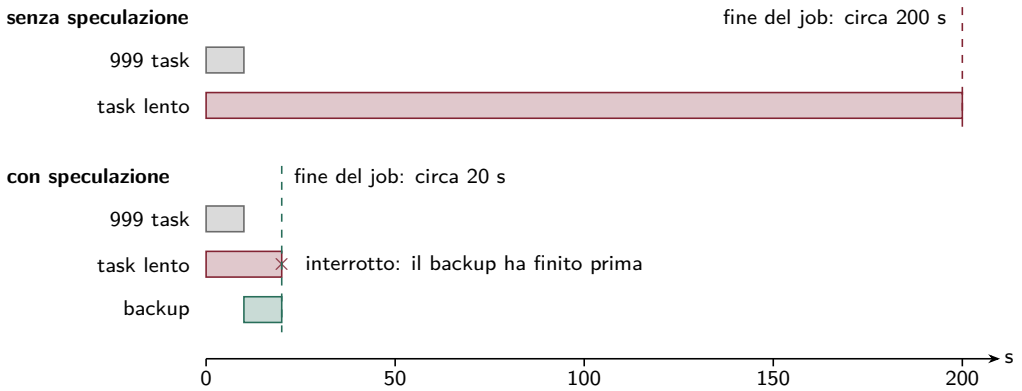


Figura 2.3. L'esempio dell'esecuzione speculativa su un asse dei tempi. Senza speculazione il job aspetta i 200 s del task lento; con la speculazione la copia di backup parte quando gli altri task hanno finito, termina prima dell'originale e il job si chiude in circa 20 s.

L'esecuzione speculativa però non è gratuita:

- **beneficio**: può ridurre drasticamente il tempo di completamento quando pochi task sono anomalmente lenti;
- **costo**: spreca risorse del cluster per copie che alla fine vengono scartate;
- **regolazione**: di solito scatta solo per task molto più lenti dei loro pari, e solo quando nel cluster c'è capacità libera.

Un solo output per task. Con la riesecuzione e con le copie di backup, lo stesso task può avere due tentativi in esecuzione insieme, ma l'output definitivo deve essere uno solo. Lo garantisce un **commit atomico**, cioè un'operazione che avviene per intero o per niente: ogni tentativo scrive in una posizione temporanea, e solo il commit rende ufficiale il suo output. Se il map task 3 gira due volte, l'originale lento e la copia di backup, il master registra l'output della copia, che finisce per prima, e ignora l'originale quando finisce anche lui. Per un reduce task decide la rinomina atomica del file temporaneo nel DFS: se due tentativi la chiedono, il file finale contiene l'output di uno solo.

La Tabella 2.3 mette a confronto questo strato di calcolo con lo strato di storage del capitolo precedente.

Tabella 2.3. Strato di storage e strato di calcolo a confronto.

Aspetto	Storage (GFS, HDFS)	Calcolo (MapReduce)
Coordinatore	master / NameNode	master / JobTracker
Chi lavora	chunkserver / DataNode	worker (map e reduce task)
Rilevamento	heartbeat	heartbeat
Recupero	ri-replicazione	riesecuzione
Concetto nuovo	—	esecuzione speculativa: lo straggler non è un guasto

2.5 Limiti di MapReduce

Le scelte che rendono MapReduce semplice e tollerante ai guasti portano anche limiti concreti:

- **uso intensivo del disco**: i dati intermedi vengono scritti su disco tra ogni map e ogni reduce, un costo pesante per gli algoritmi iterativi, come molti cicli di addestramento del machine learning;
- **struttura rigida a due stadi**: le pipeline complesse richiedono di concatenare più job MapReduce, e ciascuno paga di nuovo per intero il costo dello shuffle;
- **latenza alta**: il modello privilegia il throughput batch, non le interrogazioni interattive o a bassa latenza.

Sono questi limiti a motivare Apache Spark, con la sua esecuzione a DAG in memoria.

> **Dieci job in catena.** Un data scientist deve eseguire 10 job MapReduce in successione, in cui l'output di ciascuno è l'input del successivo, come negli algoritmi iterativi sui grafi. Il costo dominante non è il calcolo, ma l'I/O: a ogni passo l'output viene scritto nel DFS, e quindi replicato, per essere subito riletto, e ogni job rifà da capo lo shuffle. Tenere i dati in memoria tra un passo e l'altro eviterebbe gran parte di questo I/O: è l'idea da cui parte Spark.

Capitolo 3

Apache Spark e i grafi di dataflow

MapReduce ha risolto il problema della sopravvivenza: un calcolo può girare su migliaia di macchine inaffidabili e arrivare in fondo (Capitolo 2). Spark si chiede se, una volta capito come sopravvivere ai guasti, si possa anche essere veloci: tenendo i dati in memoria e ricordando come ricostruirli, invece di replicarli.

3.1 Oltre la pipeline a due stadi

I limiti di MapReduce (Sezione 2.5), cioè l'uso intensivo del disco, la struttura rigida a due stadi e la latenza alta, pesano soprattutto sui carichi iterativi, come gli algoritmi sui grafi e i cicli di addestramento del machine learning, che pagano il costo del disco e del rilancio del job a ogni iterazione. Hanno una causa comune: MapReduce è progettato attorno a dati residenti su disco e a una pipeline di forma fissa. Spark nasce per superarli.

3.1.1 Il costo dei job concatenati

Con MapReduce un'analisi a più passi è una catena di job, e l'osservazione «Dieci job in catena» (Sezione 2.5) ne ha mostrato il costo: a ogni passo una scrittura completa nel file system distribuito, con la replicazione, una lettura completa all'inizio del job dopo e un nuovo shuffle, e ogni job si lancia e si schedula da capo. Una pipeline iterativa di 10 passi fa così 10 giri completi su disco e 10 shuffle, anche se solo il primo input e l'ultimo output hanno davvero bisogno di uno storage durevole.

3.1.2 Grafi di calcolo arbitrari

Le pipeline reali hanno bisogno di catene arbitrarie di operazioni, non di un'unica forma map → reduce: filtrare, poi trasformare, poi unire con un altro dataset, poi aggregare, poi filtrare di nuovo. La forma del calcolo è un grafo generale di operazioni, non uno schema fisso a due stadi.

Definizione (DAG). Un **grafo aciclico diretto** (*Directed Acyclic Graph*, DAG) rappresenta un calcolo in modo naturale: i nodi sono le operazioni, o i dataset che producono; gli archi orientati sono le dipendenze tra i dati («l'output di questa operazione è l'input di quella»); l'assenza di cicli garantisce che il calcolo vada sempre avanti, senza mai tornare su se stesso.

In quest'ottica un job MapReduce è semplicemente un DAG con esattamente due stadi. Spark generalizza a DAG di forma e profondità qualsiasi (Figura 3.1).

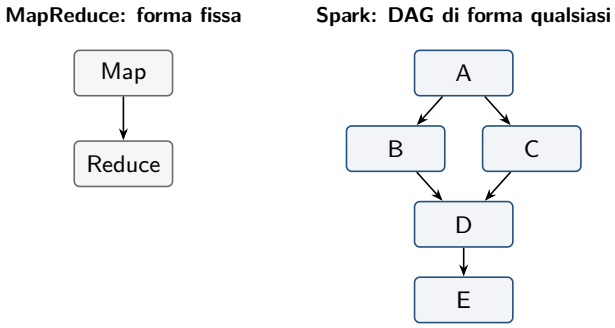


Figura 3.1. Un job MapReduce ha sempre la stessa forma, due stadi in fila; in Spark il calcolo può avere la forma di un DAG qualsiasi, con diramazioni e ricongiunzioni.

3.1.3 Esecuzione in memoria

Il secondo cambiamento di Spark è tenere i dati in memoria tra un'operazione e l'altra, quando possibile: i risultati intermedi restano in RAM e non vengono scritti nel file system distribuito, se non quando il programmatore lo chiede o quando la memoria non basta. A parità di dati, l'accesso alla RAM è circa 100 volte più veloce di quello al disco.

Spark nasce da un'osservazione: i sistemi come MapReduce non avevano un modo efficiente di riusare i dati tra più operazioni. Ne soffrivano due carichi:

- gli **algoritmi iterativi**, che scandiscono lo stesso dataset per molte iterazioni: algoritmi sui grafi come PageRank, o cicli di addestramento come la discesa del gradiente su un training set fisso. In MapReduce ogni iterazione è un job completo che rilegge l'input dal disco; in memoria il risparmio si ripete a ogni iterazione;
- le **interrogazioni interattive**: chi esplora un dataset vuole risposte in secondi, non in minuti. Se il dataset viene messo in cache in memoria una volta, le query successive evitano del tutto l'I/O su disco.

La risposta di Spark è una nuova astrazione, residente in memoria e insieme tollerante ai guasti senza affidarsi alla replicazione.

3.2 Resilient Distributed Dataset

Definizione (RDD). Un **Resilient Distributed Dataset** (RDD) è l'astrazione di dati centrale di Spark: una collezione di elementi **partizionata** tra le macchine di un cluster, **immutabile** (una volta creato, il suo contenuto non cambia mai) e **resiliente** (se una partizione va persa, viene recuperata in automatico, senza affidarsi alla replicazione).

Ogni dataset manipolato da un programma Spark, che sia letto dallo storage o derivato da un altro dataset, è un RDD. Concettualmente un RDD si comporta come una versione grande e distribuita di una semplice lista in memoria: gli si applicano operazioni che producono nuovi RDD, le trasformazioni, ma non se ne possono modificare direttamente gli elementi, e ogni «modifica» produce un RDD nuovo.

3.2.1 Perché l'immutabilità

L'immutabilità non è una restrizione arbitraria: poiché un RDD non cambia mai dopo la creazione, il modo in cui è stato creato descrive in modo completo e permanente il suo contenuto. Evita anche un'intera classe di bug di concorrenza, quelli dovuti a uno stato mutabile condiviso tra macchine. Spark registra quindi, per ogni RDD, come è stato derivato.

Definizione (lineage). Il **lineage** (letteralmente «discendenza») di un RDD è la catena registrata di trasformazioni che descrive come le sue partizioni sono state derivate dagli RDD genitori: ogni RDD ricorda da quali genitori viene e quale trasformazione lo ha prodotto.

Qui sta la differenza di fondo rispetto allo storage del Capitolo 1: se una partizione va persa, per esempio per il guasto di una macchina, Spark non ne legge una replica, come per i blocchi di GFS e HDFS, ma la ricalcola dal lineage.

3.2.2 Partizioni

Come i blocchi dello storage e i task di MapReduce, anche gli RDD sono divisi in **partizioni**. Ogni partizione è un sottoinsieme dei dati, che risiede su una macchina alla volta, o è elaborato da una macchina alla volta. Le operazioni su un RDD si applicano in modo indipendente e in parallelo a ciascuna partizione, quindi il numero di partizioni fissa il grado massimo di parallelismo.

Le partizioni sono imparentate con i blocchi (Sezione 1.2), ma sono un'altra cosa. Un RDD creato da un file HDFS ha di solito una partizione per blocco del file; ma lo si può ripartizionare esplicitamente, o creare da sorgenti che non sono file.

3.2.3 Creare un RDD

Un RDD non è un sistema di storage, ma un'astrazione di calcolo costruita sopra lo storage esistente. Nasce in due modi:

```
# da uno storage distribuito (per esempio HDFS)
rdd = sc.textFile("hdfs://path/to/data")
# da una collezione in memoria
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

Qui `sc` (lo SparkContext) è il punto di accesso a Spark del **driver**, il programma dell'utente, che crea gli RDD e riceve i risultati. Con `textFile` le partizioni derivano dai blocchi del file; con `parallelize` la collezione locale del driver viene divisa in partizioni e distribuita al cluster.

3.2.4 Caching e persistenza

Una funzione centrale per le prestazioni è la possibilità di tenere esplicitamente un RDD in memoria (*cache*) tra un uso e l'altro:

```
rdd = sc.textFile("hdfs://path/to/data")
rdd.cache()   # da tenere in memoria
```

Senza caching Spark potrebbe dover ricalcolare l'RDD da zero, tramite il lineage, ogni volta che viene riusato; con il caching le partizioni calcolate restano in memoria, pronte per le operazioni successive. È ciò che rende possibili i carichi iterativi e interattivi della Sezione 3.1. Spark offre diversi livelli di persistenza, che bilanciano velocità e capacità (Tabella 3.1); la scelta dipende da quanto è grande il dataset rispetto alla memoria del cluster.

Tabella 3.1. Livelli di persistenza di un RDD.

Livello	Comportamento
Solo memoria	il più veloce; le partizioni che non ci stanno vengono ricalcolate quando servono
Memoria e disco	le partizioni in eccesso finiscono sul disco locale invece di essere scartate
Solo disco	nessuna pressione sulla memoria, al prezzo di un accesso alla velocità del disco

3.3 Trasformazioni e azioni

Ogni operazione che un programma Spark esegue su un RDD appartiene a una e una sola di due categorie.

Definizione (trasformazione e azione). Una **trasformazione** prende uno o più RDD e produce un nuovo RDD, che descrive un dataset derivato. Un'**azione** fa partire il calcolo e produce un risultato fuori dall'astrazione degli RDD: lo restituisce al driver, oppure lo scrive nello storage.

Le trasformazioni sono funzioni deterministiche sui dati, come la funzione map del Capitolo 2, e per l'immutabilità non modificano mai l'RDD di partenza:

```
rdd1 = sc.textFile("data.txt")
rdd2 = rdd1.filter(lambda line: "error" in line)
# rdd1 non cambia; rdd2 descrive il risultato filtrato
```

`rdd1` resta esattamente com'era, e `rdd2` è un dataset distinto, appena descritto: una catena di trasformazioni produce soltanto una catena di descrizioni di RDD. La Tabella 3.2 raccoglie le trasformazioni e le azioni più comuni.

Tabella 3.2. Trasformazioni e azioni più comuni.

Operazione	Effetto
Trasformazioni	
`map(f)`	applica `f` a ogni elemento
`filter(f)`	tiene gli elementi per cui `f` è vera
`flatMap(f)`	applica `f` e appiattisce le liste risultanti
`distinct()`	elimina i duplicati
`union(other)`	unisce all'RDD un altro RDD
`groupByKey()`	raggruppa i valori per chiave
`reduceByKey(f)`	aggrega con `f` i valori di ogni chiave
`join(other)`	unisce per chiave due RDD di coppie chiave-valore
Azioni	
`count()`	restituisce il numero di elementi
`collect()`	restituisce tutti gli elementi al driver
`take(n)`	restituisce i primi `n` elementi
`reduce(f)`	aggrega tutti gli elementi con `f`
`saveAsTextFile(path)`	scrive il contenuto dell'RDD nello storage

Esempio (word count in Spark). L'esempio canonico del Capitolo 2, scritto con gli RDD:

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

In `reduceByKey` la funzione riceve due conteggi `a` e `b` della stessa parola e li somma: Spark la applica a due a due, come il `reduce` funzionale, finché per ogni parola resta un solo valore. `flatMap`, `map` e `reduceByKey` sono tutte trasformazioni, e a questo punto non è stato calcolato nulla: `lines`, `words`, `pairs` e `counts` sono solo descrizioni di RDD, che dicono come calcolare il risultato, non il risultato. Il calcolo parte solo con un'azione:

```
result = counts.collect()   # azione: esegue l'intera catena
```

Questa riga fa leggere l'input da HDFS, eseguire `flatMap`, `map` e `reduceByKey`, compreso lo shuffle necessario, e infine raccogliere i risultati nel driver.

Due reduce diversi. I nomi possono confondere. In Spark `reduce(f)` è un'azione: aggrega tutti gli elementi di un RDD in un unico valore, restituito al driver. `reduceByKey(f)` è invece una trasformazione: aggrega i valori di ogni chiave e produce un nuovo RDD di coppie. Nessuna delle due coincide con la funzione reduce di MapReduce, che l'utente scrive e il framework chiama una volta per chiave: nel word count il suo ruolo lo svolge `reduceByKey`.

3.4 Valutazione pigra e DAG di esecuzione

Definizione (valutazione pigra). La **valutazione pigra** (*lazy evaluation*) è il principio per cui le trasformazioni vengono registrate come un piano, non eseguite subito: chiamare `map`, `filter` o qualunque altra trasformazione aggiorna soltanto la descrizione interna del calcolo, e nessuna partizione viene letta, trasformata o scritta finché un'azione non lo impone.

È ciò che si è visto nel word count: fino a `collect()` non viene calcolato nulla. Per questo un RDD è un riferimento logico (*handle*): descrive «un dataset, partizionato così, derivato da questo calcolo», e il suo contenuto può non esistere finché un'azione non ne forza il calcolo. La pigrizia non rende le trasformazioni gratuite, ne rimanda soltanto il costo all'azione; in cambio, quando l'azione arriva, Spark conosce l'intera catena e la può ottimizzare come un tutto, invece di eseguire ogni operazione appena viene chiamata.

3.4.1 Il piano logico e il DAG

Mentre le trasformazioni si concatenano, Spark costruisce un **piano di esecuzione logico**: ogni RDD registra un riferimento ai propri RDD genitori e la trasformazione che lo ha prodotto. Ne risulta una catena, o più in generale un grafo, di RDD che puntano a come sono stati derivati: è esattamente il lineage della Sezione 3.2, visto qui nel suo ruolo di piano di esecuzione.

Questa struttura forma un DAG: ogni RDD è un nodo e ogni trasformazione un arco orientato dagli RDD genitori al nuovo RDD; le operazioni che combinano più RDD, come `join` e `union`, creano nodi con più archi entranti. Nel word count il DAG è una catena, `lines` → `words` → `pairs` → `counts`, e l'intero grafo esiste nella memoria di Spark prima che venga chiamata `collect()`: è il piano logico, non ancora eseguito.

3.4.2 Ottimizzazioni

Vedere tutto il DAG prima dell'esecuzione permette due ottimizzazioni:

- la **fusione degli operatori** (*operator fusion*, o *pipelining*): concatenare operazioni compatibili, per non materializzare, cioè calcolare e salvare per intero, risultati intermedi inutili;

- **evitare calcoli inutili**: saltare le parti del DAG che non servono al risultato; se un'azione ha bisogno solo di una parte dei dati, come `take(5)`, a volte Spark riesce a non calcolare più del necessario.

Il pipelining vale per le trasformazioni che si possono applicare a un elemento senza aspettare gli altri. In

```
words.map(f).filter(g).map(h)
```

senza pipelining Spark applicherebbe `f` a tutti gli elementi della partizione e salverebbe il risultato, poi `g` a tutti, poi `h`: tre passate e due risultati intermedi. In pipeline prende il primo elemento e gli applica `f`, `g` e `h`, poi passa al secondo, e così via: una passata sola, senza risultati intermedi. Resta separato ciò che conta: ogni elemento passa ancora per `f`, `g` e `h`, nello stesso ordine, quindi il risultato non cambia. Nei job concatenati di MapReduce, invece, ogni risultato intermedio veniva scritto per intero in HDFS prima del passo dopo.

3.4.3 Dal DAG logico all'esecuzione

Quando finalmente viene chiamata un'azione, il **DAG scheduler**, il componente di Spark che coordina l'esecuzione, analogo al master di MapReduce, divide il DAG in **stage** (fasi) e ogni stage in **task**, uno per partizione, che distribuisce sul cluster come i task di MapReduce (Sezione 2.2).

Definizione (stage). Uno **stage** è un gruppo di operazioni che si possono eseguire insieme, senza bisogno di rimescolare i dati in rete tra l'una e l'altra. Le operazioni di uno stage vengono eseguite in pipeline; un nuovo stage comincia ogni volta che il DAG richiede uno shuffle, cioè una redistribuzione dei dati tra le partizioni prima che l'operazione successiva possa procedere.

I confini tra gli stage non sono arbitrari: cadono esattamente dove un'operazione non si può calcolare con i soli dati locali della partizione. Operazioni come `map` o `filter` si concatenano liberamente, perché non hanno bisogno di dati di altre partizioni; operazioni come `reduceByKey` o `join` devono raggruppare di nuovo i dati tra le partizioni, esattamente come lo shuffle di MapReduce (Sezione 2.3).

Esempio (word count esteso). Si aggiunge un passo al word count: dopo il conteggio si scartano le parole rare.

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
frequent = counts.filter(lambda kv: kv[1] > 100)
result = frequent.collect()   # azione: esegue l'intera catena
```

Ogni elemento di `counts` è una coppia (parola, conteggio), qui chiamata `kv`: `kv[0]` è la parola e `kv[1]` il conteggio, quindi il filtro tiene le parole che compaiono più di 100 volte. Sono cinque passi concatenati, eseguiti come un

unico DAG quando si chiama `collect()`.

3.5 Dipendenze strette e ampie

Una **dipendenza** descrive il rapporto tra le partizioni di un RDD figlio e le partizioni degli RDD genitori da cui dipendono: è il lineage visto partizione per partizione. Il tipo di dipendenza determina quanto costa eseguire una trasformazione e se questa impone un nuovo stage.

Definizione (dipendenza stretta e ampia). Una dipendenza è **stretta** (*narrow*) quando ogni partizione dell'RDD figlio dipende da un numero piccolo e fisso di partizioni dell'RDD genitore, di solito una sola. È **ampia** (*wide*) quando una partizione del figlio può dipendere da dati di più partizioni del genitore, potenzialmente di tutte.

`map`, `filter` e `flatMap` sono strette: ogni partizione di output si calcola solo dalla partizione di input corrispondente, e nessun dato si sposta tra partizioni che stanno su macchine diverse. `groupByKey`, `reduceByKey` e `join` sono ampie: per produrre una sola partizione di output possono servire dati sparsi in tutte le partizioni di input (Figura 3.2). Nel word count, la partizione di `counts` che contiene «the» ha bisogno delle coppie `(the,1)` di tutte le partizioni di `pairs`, perché «the» compare un po' ovunque. È la stessa struttura del movimento di dati all-to-all dello shuffle di MapReduce (Sezione 2.3).

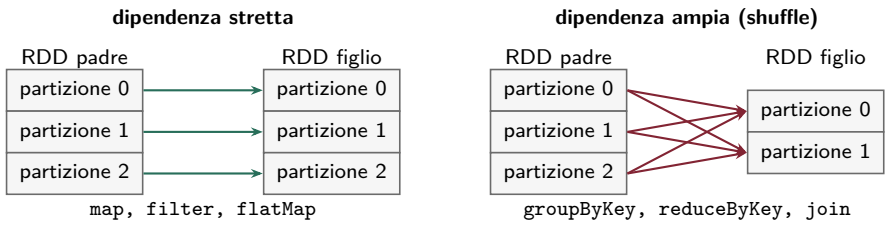


Figura 3.2. Dipendenze strette e ampie. Nella stretta ogni partizione del figlio dipende da una sola partizione del padre, e il calcolo resta locale; nell'ampia ogni partizione del figlio può ricevere dati da tutte quelle del padre, e serve uno shuffle in rete.

3.5.1 Pipeline e shuffle

La distinzione è il meccanismo preciso dietro ai confini degli stage.

- Le dipendenze strette permettono di concatenare le operazioni in pipeline in sicurezza: una catena di trasformazioni strette si calcola tutta dentro una partizione, su una macchina, in una passata. È il pipelining della Sezione 3.4, e ora si vede perché è possibile: ogni passo è una dipendenza stretta.
- Le dipendenze ampie richiedono uno shuffle: non si può calcolare `reduceByKey` senza prima raggruppare di nuovo i dati per chiave in tutto il cluster, con partizionamento, trasferimento in rete e raggruppamento, come in MapReduce.

Spark non evita il costo dello shuffle: lo confina ai soli punti del DAG in cui è inevitabile, invece di pagarlo a ogni job come una catena di job MapReduce.

Si può quindi dire con precisione dove finisce uno stage: uno stage è una sequenza massimale di trasformazioni con dipendenze strette, eseguibili in pipeline, e un nuovo stage comincia esattamente a ogni dipendenza ampia, perché per calcolarla serve che l'output dello shuffle sia disponibile.

Esempio (gli stage del word count esteso). Il DAG scheduler costruisce il piano del word count esteso in quattro passi (Figura 3.3):

1. risale il lineage all'indietro, a partire dall'RDD dell'azione: `frequent`, `counts`, `pairs`, `words`, `lines`;
2. raggruppa le catene di dipendenze strette in stage: lo stage 1 legge `lines` e applica `flatMap` e `map`, lo stage 2 applica `reduceByKey` e `filter`; il confine cade a `reduceByKey`, che è ampia;
3. ordina gli stage: lo stage 2 parte solo quando lo stage 1 ha scritto tutto l'output dello shuffle;
4. dentro ogni stage crea un task per partizione: con tre partizioni per RDD, tre task per lo stage 1 e tre per lo stage 2.

La catena di cinque passi diventa così due soli stage.

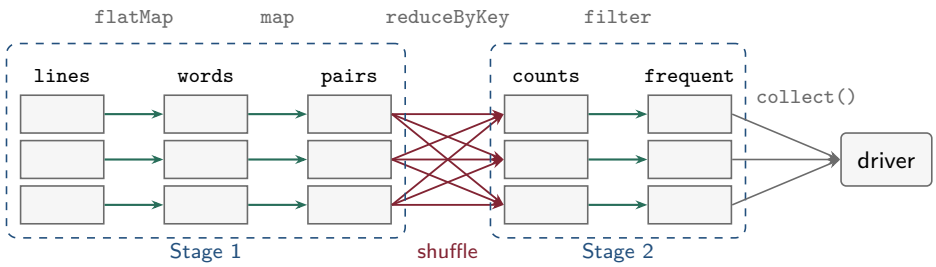


Figura 3.3. Gli stage del word count esteso, con tre partizioni per RDD. Le trasformazioni strette (in verde) stanno in pipeline dentro uno stage; `reduceByKey` (in rosso) è una dipendenza ampia, quindi uno shuffle e l'inizio del secondo stage. L'azione `collect()` riporta i risultati al driver.

Per ogni task lo scheduler preferisce una macchina vicina ai dati, come in GFS, HDFS e MapReduce (Sezione 1.5). Nel primo stage, che legge da HDFS, i task vanno sulle macchine che ospitano i blocchi corrispondenti, esattamente come i map task; negli stage successivi Spark preferisce le macchine che hanno già in memoria le partizioni genitrici, magari in cache. La Tabella 3.3 mette a confronto l'esecuzione di MapReduce e quella di Spark.

Tabella 3.3. L'esecuzione in MapReduce e in Spark.

Aspetto	MapReduce	Spark
Forma	due stadi fissi, map poi reduce	DAG di forma e profondità qualsiasi
Quando si calcola	subito, mentre ogni job gira	rimandato finché non si chiama un'azione
Visibilità	un job alla volta, isolato	l'intera catena di trasformazioni, prima dell'esecuzione
Dati intermedi	su disco, e nel DFS tra un job e l'altro	in memoria e in pipeline, dove possibile
Riuso dei dati	riletti dal disco a ogni iterazione	in cache in memoria
Shuffle	in ogni job, tra map e reduce	solo alle dipendenze ampie

3.5.2 Il costo delle dipendenze ampie

Le dipendenze ampie hanno gli stessi costi dello shuffle di MapReduce:

- consumano banda di rete in proporzione al volume dei dati redistribuiti;
- richiedono I/O su disco: l'output del lato map di uno shuffle viene sempre scritto sul disco locale, come l'output dei map in MapReduce, così dopo un guasto basta rileggerlo; quando la memoria scarseggia, Spark riversa su disco anche i buffer con cui ordina e aggrega;
- sono un punto di sincronizzazione: lo stage dopo aspetta tutte le partizioni dello stage prima, come nel word count esteso.

Poiché sono la voce di costo dominante, i programmi Spark si scrivono in modo da ridurne numero e impatto:

- si preferisce `reduceByKey` a `groupByKey` seguito da un'aggregazione separata, perché `reduceByKey` può aggregare parzialmente in locale prima dello shuffle, proprio come un combiner di MapReduce (Sezione 2.2.5);
- dove possibile, si spostano le operazioni strette, come filtri e map, prima di quelle ampie, per ridurre il volume di dati da rimescolare.

3.6 Lineage e tolleranza ai guasti

Come per lo storage e per MapReduce, i guasti su un cluster sono routine: una macchina che ospita partizioni di uno o più RDD può andare in crash a metà calcolo. Le risposte viste finora sono due: lo storage replica i dati su più macchine (Capitolo 1), MapReduce riesegue i task falliti a partire dall'input originale (Capitolo 2). Spark costruisce su entrambe e ricalcola soltanto le partizioni perse, usando il lineage (Sezione 3.2): lo stesso DAG costruito per l'esecuzione, usato ora per il recupero.

3.6.1 Ricalcolo di una partizione

Quando una macchina si guasta, i task che vi giravano falliscono, e il DAG scheduler, che ha il lineage, individua esattamente quali partizioni sono andate perse. Per ciascuna:

1. individua dal lineage le partizioni genitrici da cui dipende;
2. se queste sono ancora disponibili (in memoria, su disco, o a loro volta ricalcolabili), lancia un task che ricalcola la partizione persa riapplicando la trasformazione;
3. se manca anche una partizione genitrice, risale ancora la catena del lineage, finché non trova dati disponibili.

I task di recupero usano la stessa struttura di stage e dipendenze dell'esecuzione normale: non c'è un sistema di recupero separato, gestito come caso speciale.

Esempio (una partizione persa). Si consideri la catena RDD originale → A → B → C, dove A si ottiene con `map`, B con `filter` e C con `reduceByKey`, e supponiamo che vada persa una partizione di B, perché la sua macchina si è guastata (Figura 3.4). Spark risale il lineage: la partizione di B dipende da una partizione nota di A. Se questa è ancora disponibile, basta riapplicarle `filter` per ricreare la partizione persa, senza toccare nessun'altra partizione.

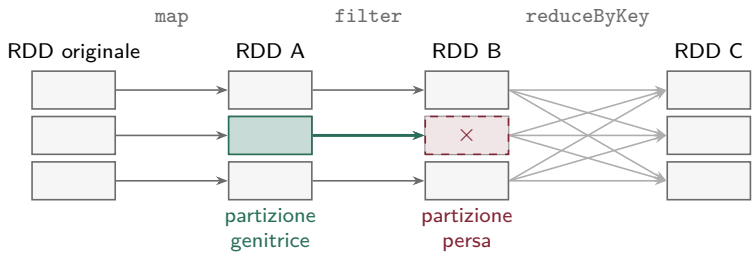


Figura 3.4. Ricalcolo dal lineage. Si perde una partizione di B (in rosso); la si ricostruisce riapplicando `filter` alla sola partizione di A da cui dipende (in verde), senza toccare le altre.

3.6.2 Tre strategie a confronto

La Tabella 3.4 mette a confronto le tre strategie di tolleranza ai guasti viste nel modulo. Rispetto a MapReduce, l'approccio di Spark è una generalizzazione diretta del «basta riprovare», applicata partizione per partizione lungo un DAG arbitrariamente lungo. Rispetto alla replicazione è un vero compromesso, non un miglioramento netto: si risparmia spazio, ma si paga il ricalcolo.

Tabella 3.4. Tre strategie di tolleranza ai guasti.

Aspetto	GFS e HDFS	MapReduce	Spark
Meccanismo	replicazione fisica, di solito 3×	riesecuzione dei task falliti	ricalcolo dal lineage
Si ricalcola	nulla: si legge una replica	il task fallito, per intero	solo le partizioni perse, seguendo il lineage
Spazio in più	3 volte il volume dei dati	—	minimo: il grafo delle trasformazioni
Recupero	veloce: si legge una replica esistente	—	si ricalcola dal lineage, lento se la catena è lunga
Più passi	—	il guasto di ogni job si gestisce a parte	un'unica catena di lineage copre tutta la pipeline

Replicare ogni RDD intermedio sarebbe proibitivo: una pipeline di dieci trasformazioni richiederebbe dieci dataset replicati e sincronizzati. Il lineage invece memorizza per ogni RDD pochi metadati, in quantità costante qualunque sia la dimensione dei dati.

3.6.3 Catene lunghe e checkpointing

Il recupero dal lineage ha però un limite. Se la catena è molto lunga, ricalcolare una partizione persa verso la fine può richiedere di ricalcolare gran parte di tutto ciò che la precede: il costo cresce con la profondità del DAG, ed è un problema soprattutto per gli algoritmi iterativi, dove lo stesso RDD viene trasformato di nuovo a ogni iterazione.

Per limitarlo Spark permette il **checkpointing** esplicito. Un checkpoint scrive il contenuto attuale di un RDD su uno storage durevole e replicato, come HDFS, nello spirito della replicazione del Capitolo 1. Dopo il checkpoint la catena del lineage di fatto riparte da lì: un ricalcolo futuro risale solo fino al checkpoint, non fino all'input originale. Con un checkpoint ogni 10 iterazioni, per esempio, una partizione persa all'iterazione 95 si ricalcola a partire dall'iterazione 90, non dalla prima. È un compromesso deliberato: si paga subito un costo di storage e di replicazione, in cambio di un costo di ricalcolo limitato in seguito.

3.7 Limiti degli RDD

Gli RDD restano un'astrazione di basso livello: le interrogazioni di tipo relazionale diventano prolisse, gli RDD non conoscono la struttura dei dati (schemi e colonne), che permetterebbe altre ottimizzazioni, e il modello batch non gestisce flussi di dati continui. Sistemi più recenti vanno oltre: Apache Flink per gli stream, Ray per i grafi di task dinamici dell'AI distribuita, Delta Lake per lo storage lakehouse.